

Yazılım Notlarım

Ömer Faruk Yavuz

November 2025

Introduction

Günlük yazılım pratiğinde verdiğimiz kararların çoğu, aslında başkalarının daha önce aldığı kararların tekrarından ibarettir: “Bu projede hep microservice kullandık”, “Frontend mutlaka şu framework ile yazılır”, “Veritabanı olarak X seçmek en iyisi” gibi varsayımlar çoğu zaman sorgulanmadan kabul edilir. Bu yaklaşım kısa vadede işleri hızlandırıyor gibi görünse de uzun vadede gereksiz karmaşıklık, taşınması zor mimariler ve kök nedeni anlaşılmamış problemler doğurur.

First principle thinking tam burada devreye girer. Temelde şu soruyu sorar: “Bu problemi, eldeki tüm varsayımları bir kenara bırakıp, en temel gerçeklere inerek nasıl çözerdim?” Yazılımda bu; problemin doğasını, kısıtlarını, verisini ve başarımlar gereksinimlerini çıplak hâliyle anlamayı, ardından araçları ve teknolojileri *bu* anlayışın üzerine seçmeyi ifade eder. Yani önce framework değil, önce fizik; önce “React mı?” değil, önce “Kullanıcının çözmeye çalıştığı gerçek problem ne?” sorusu gelir.

Bu metnin amacı; yazılım geliştiricinin günlük kararlarını alırken kullanabileceği bir düşünme çerçevesi sunmaktır. Tasarım kalıpları, popüler mimariler veya trend teknolojilerden bağımsız olarak, problemi temel bileşenlerine ayırma, varsayımları test etme ve sıfırdan mantık kurma alışkanlığımı sistematik hâle getirmeyi hedefler. Böylece okur, hem kod yazarken hem hata ayıklarken hem de mimari ve ürün kararları alırken tekrarlanabilir, rasyonel bir zihinsel süreç geliştirebilir.

OOP (Object-Oriented Programming) Principles

Dört Temel OOP Prensibi

Tüm dillerde ortaktır (Java, C#, TypeScript, Kotlin, Swift vb.):

1. **Encapsulation** (Kapsülleme)
2. **Abstraction** (Soyutlama)
3. **Inheritance** (Kalıtım)
4. **Polymorphism** (Çok Biçimlilik)

Aşağıda her bir prensip Angular/TypeScript perspektifiyle açıklanmıştır.

1. Encapsulation — Veriyi koru, erişimi kontrol et

Amaç

Bir nesnenin verisini dışarıdan korunmuş tutmak ve sadece kontrollü yollarla erişilmesini sağlamak.

Angular Örneği

```
export class User {  
  private password: string;  
  
  constructor(private email: string, pass: string) {  
    this.password = pass;  
  }  
  
  getEmail() {  
    return this.email;  
  }  
  
  checkPassword(p: string) {  
    return p === this.password;  
  }  
}
```

Özet

“Veriyi sakla, dışarıya kontrollü API ver.”

2. Abstraction — Gereksiz detayları sakla

Amaç

Sistemin gereksiz karmaşıklığını gizlemek ve kullanıcıya yalnızca ihtiyaç duyduğu yüzeyi göstermek.

Angular Servis Soyutlaması

```
export abstract class Auth {  
  abstract login(email: string, pass: string): Observable<User>;  
}  
  
@Injectable({ providedIn: 'root' })  
export class FirebaseAuthService extends Auth {  
  login(email: string, pass: string) { /* Firebase logic */ }  
}  
  
@Injectable({ providedIn: 'root' })  
export class LocalAuthService extends Auth {  
  login(email: string, pass: string) { /* local logic */ }  
}
```

Özet

“Sadece gerekeni göster, gerisini sakla.”

3. Inheritance — Ortak davranışı miras almak

Amaç

Ortak özellik ve davranışları üst sınıfta toplamak; alt sınıfların bu ortak yapıyı miras almasını sağlamak.

Angular Base Component Örneği

```
export class BaseFormComponent {
  isLoading = false;

  start() { this.isLoading = true; }
  end()    { this.isLoading = false; }
}

@Component({...})
export class LoginComponent extends BaseFormComponent {
  submit() {
    this.start();
    this.end();
  }
}
```

Özet

“Ortak şeyleri yukarı koy, aşağıya miras bırak.”

4. Polymorphism — Aynı metot ismi, farklı davranış

Amaç

Aynı arayüzü paylaşan farklı sınıfların metotları farklı şekilde uygulamasını sağlamak.

Angular Örneği

```
interface NotificationService {
  notify(msg: string): void;
}

class SnackbarService implements NotificationService {
  notify(msg: string) { /* Material snackbar */ }
}

class ToastrServiceImpl implements NotificationService {
  notify(msg: string) { /* Toastr popup */ }
}

function showMessage(service: NotificationService) {
  service.notify('Hello!');
}
```

Özet

“Aynı isim → farklı davranış.”

Karşılaştırmalı Tablo

Prensip	Kısa Tanım	En Kısa Özet
Encapsulation	Veriyi gizle, kontrollü eriştir	“Şifreyi kimse görmesin.”
Abstraction	Gereksiz detayı gizle	“Kullanıcı pedala bassın, motoru bilmesin.”
Inheritance	Ortak davranışı miras al	“BaseForm → LoginForm bunu kullansın.”
Polymorphism	Aynı arayüz, farklı davranış	“notify() her sistemde başka çalışsın.”

Priority Queue (Öncelikli Kuyruk)

Priority Queue, normal bir kuyruğun aksine elemanları **giriş sırasına göre değil, önceliğe göre** çıkaran veri yapısıdır. FIFO değildir; **önceliği en yüksek** (veya Min-Heap ise en düşük) eleman her zaman ilk çıkar. Bir eleman (değer, öncelik) ikilisiyle kuyruğa eklenir. Çıkarma (dequeue) işleminde sıradaki eleman, **önceliği en yüksek olan** elemandır.

Priority Queue çoğunlukla **Min-Heap** veya **Max-Heap** ile implemente edilir. Bunun sebebi performanstır:

İşlem	Zaman Karmaşıklığı
enqueue (ekleme)	$O(\log n)$
dequeue (çıkarma)	$O(\log n)$
peek (en yüksek/ düşük öncelik)	$O(1)$

Array ile benzer performans elde edilemez; bu nedenle Heap idealdir.

Gerçek Hayat Kullanım Alanları

- **UI / Frontend** — React Fiber Scheduler (min-heap), animasyon öncelikleri
- **Tarayıcı** — event scheduling, frame task ordering
- **Backend / Node.js** — job queue, task runners, background workers
- **Oyun / Grafik** — Dijkstra, A* pathfinding (tamamen priority queue tabanlı)
- **Rate Limiting** — kritik istekleri öne alma

Basit Kod Örneği

```
const pq = new MinPriorityQueue();

pq.enqueue("low", 5); // düşük öncelik
pq.enqueue("high", 1); // yüksek öncelik

console.log(pq.dequeue().element);
// output: "high" (önce yüksek öncelik çıkar)
```

Kısa Özet

“Priority Queue → sırayı girişe göre değil, önceliğe göre yönetir. Hız için Heap kullanır.”

Linked List

Tanım

Linked List, her biri bir sonraki düğüme işaret eden (pointer) **node**'lardan oluşan doğrusal bir veri yapısıdır. Her node iki parçadan oluşur:

- **data** — saklanan değer
- **next** — bir sonraki düğüme işaret eden pointer

Türleri

Singly Linked List

Her node yalnızca bir sonraki node'a işaret eder. Son node'un **next** pointer'ı null değerini gösterir.

Doubly Linked List

Her node iki pointer içerir:

- **prev** — önceki node
- **next** — sonraki node

Son node'un **next** pointer'ı null değerini gösterir.

Circular Linked List

Son node'un **next** pointer'ı listenin ilk node'una döner. Liste dairesel biçim alır.

Zaman Karmaşıklığı

Erişim (Access)	$O(n)$
Arama (Search)	$O(n)$
Ekleme (Insert)	$O(1)$
Silme (Remove)	$O(1)$

Notlar

- Rastgele erişim yoktur; elemanlar sırayla gezinerek bulunur.
- Ekleme ve silme işlemleri pointer değişimi ile yapıldığı için çok hızlıdır.
- Büyük dinamik listeler, queue/stack implementasyonları ve memory-sensitive durumlarda kullanılır.

Linked List'in Gerçek Hayat Kullanım Alanları

Linked List çoğu zaman FE/BE uygulamalarında doğrudan kodlanan bir veri yapısı şeklinde kullanılmaz; ancak modern teknolojilerin büyük kısmı arka planda Linked List mantığını kullanır. Aşağıdaki örnekler gerçek sistemlerde birebir kullanıldığı alanlardır.

1. Undo / Redo Mekanizmaları (Photoshop, VSCode, Word, Figma)

Her kullanıcı işlemi bir node olarak saklanır. `prev` ve `next` pointer'ları arasında gezinerek **undo**/**redo** işlemleri yapılır.

$$Edit_1 \leftrightarrow Edit_2 \leftrightarrow Edit_3$$

- Ctrl+Z → önceki node
- Ctrl+Shift+Z → sonraki node

Gerçek uygulamalarda **doubly linked list** kullanılır.

2. Tarayıcı Geçmişi (Browser History)

Web tarayıcılarında ileri/geri gezinme işlemleri doğrudan çift bağlı liste (doubly linked list) mantığıyla yapılır.

$$Page_1 \leftrightarrow Page_2 \leftrightarrow Page_3$$

3. LRU Cache (React Query, TanStack Query, Redis, OS Cache)

LRU cache yapıları genellikle aşağıdaki kombinasyonu kullanır:

- Hash Map (O(1) erişim)
- Doubly Linked List (O(1) ekleme/silme)

En son erişilen eleman listenin başına taşınır.

$$MostRecent \leftrightarrow \dots \leftrightarrow LeastRecent$$

4. Müzik / Video Playlist (Spotify, YouTube)

Bir sonraki veya önceki medya ögesine geçiş:

$$A \leftrightarrow B \leftrightarrow C$$

Liste dinamik olduğundan Linked List yüksek performans sağlar.

5. İşletim Sistemi Süreç Planlayıcıları (Process Scheduling)

Linux ve diğer işletim sistemleri:

- process list
- ready queue
- waiting queue

gibi yapıları linked list ile saklar.

6. Oyun Motorları (Unity, Unreal, Godot)

Aşağıdaki yapılar sıklıkla linked list temellidir:

- render queue
- physics update chain
- animation update list

Node ekleme/silme maliyetlerinin düşük olması tercih sebebidir.

7. React Fiber Architecture

React Fiber, tamamen linked list tabanlı bir yapı kullanır.

Her fiber node şu pointer'lara sahiptir:

child → *sibling* → *return*

8. Job Queue / Message Queue / Task Runner

Node.js task queue, RabbitMQ, Kafka, Celery, BullMQ gibi teknolojiler arka planda FIFO yapıları için linked list kullanır.

9. Derleyici Tasarımı (Compiler Design)

Lexer tarafından üretilen token'lar bir linked list içinde tutulur ve parser bu liste üzerinde ilerler.

10. Özelleştirilmiş Veri Yapıları (Editor, Canvas, 3D Engines)

Aşağıdaki sistemlerde sıkça linked list kullanılır:

- collaborative editors
- drawing/canvas araçları
- scene graph / object tree yapıları

Node manipülasyonlarının yoğun olduğu durumlarda performans avantajı sunduğu için tercih edilir.

Özet

Linked List özellikle şu durumlar için uygundur:

- Undo/Redo yapıları
- Navigasyon (geri/ileri)
- LRU cache
- React Fiber
- Queue/Scheduler sistemleri
- Playlist yapıları
- Game engine update loops

Eleman ekleme/silme işlemlerinin yoğun olduğu yapılarda Linked List **yüksek performans** sağlar.

Linked List vs Array

1. Bellekte Tutulma Farkı

Array (Dizi)

Array yapısı bellekte **ardışık (contiguous)** biçimde tutulur. Bütün elemanlar yan yana depolanır.

$$a_0 \ a_1 \ a_2 \ a_3 \ \dots$$

Bu nedenle herhangi bir elemana erişim çok hızlıdır ($O(1)$).

Linked List

Linked List yapısında node'lar bellekte **dağınık (non-contiguous)** tutulur. Her node bir sonraki node'a **pointer** ile işaret eder.

$$Node_1 \rightarrow Node_2 \rightarrow Node_3 \rightarrow \dots$$

Bellek düzenli değildir; ilişki pointer'lar üzerinden sağlanır.

2. Erişim (Access) Zaman Karmaşıklığı

- **Array:** Doğrudan indeksleme ile $O(1)$

$$a[i] = constanttime$$

- **Linked List:** Sıralı gezinme ile $O(n)$

$$head \rightarrow next \rightarrow \dots \rightarrow i$$

3. Ekleme / Silme (Insert / Delete)

Array

Bir elemanı ortaya eklemek veya silmek için diğer elemanların kaydırılması gerekir. Bu nedenle işlem maliyeti $O(n)$ 'dir.

$$1 \ 2 \ 3 \Rightarrow arayaekleme \Rightarrow 1 \ X \ 2 \ 3$$

Linked List

Sadece pointer'ların güncellenmesi yeterlidir. Bu işlem $O(1)$ 'dir.

$$A \rightarrow B \Rightarrow A \rightarrow X \rightarrow B$$

4. Pointer vs Index

- Array: **index tabanlıdır**. Elemanlara indeks ile erişilir.
- Linked List: **pointer tabanlıdır**. Her node bir sonrakini işaret eder.

5. Özet Karşılaştırma Tablosu

Özellik	Array	Linked List
Bellek Yerleşimi	Ardışık	Dağınık
Erişim	$O(1)$	$O(n)$
Ekleme	$O(n)$	$O(1)$
Silme	$O(n)$	$O(1)$
Pointer	Yok	Var
Kullanım Alanı	Rastgele erişim	Sık ekleme/silme

Sonuç

Linked List, bir **array değildir**. Mantığı tamamen farklıdır:

$$Array = indeks \quad \quad \quad LinkedList = pointer$$

Array rasgele erişim için idealdir; Linked List ise sık ekleme/silme yapılan yapılarda üstün performans sağlar.

Karşılaştırma Tablosu

Özellik	Singly Linked List	Doubly Linked List	Circular Linked List
Son node	null gösterir	null gösterir	head 'i gösterir
Başlangıç noktası	sabittir	sabittir	herhangi bir node olabilir
Traversel yönü	tek yön	çift yön	tek/çift yön ama döngüsel
Bitiş durumu	null ile anlaşılır	null ile anlaşılır	özel koşul gerekir
Uygunluk alanı	basit listeler	undo/redo, history	ring, round-robin, loop
Node ekleme/silme	$O(1)$	$O(1)$	$O(1)$
Bellek maliyeti	düşük	daha yüksek	orta

Linked List Türlerinin Kullanım Alanları

Aşağıda üç temel linked list türünün hangi durumlarda tercih edildiği, gerçek hayat örnekleriyle birlikte özetlenmiştir.

1. Singly Linked List — Ne Zaman Kullanılır?

Singly Linked List, en basit ve en düşük bellek tüketimine sahip linked list türüdür. Tek yönde gezinme yeterliyse idealdir.

Kullanım Senaryoları

- Belleğin kısıtlı olduğu sistemlerde (embedded, düşük seviyeli programlama).
- Sadece ileri yönde traversal gereken yapılarda.
- Ekleme ve silme operasyonlarının sık olduğu yapılarda.
- FIFO mantığında basit kuyruk (queue) implementasyonlarında.
- Streaming data veya ardışık işlem zincirlerinde.

Gerçek Hayat Örnekleri

- Basit queue yapıları.
- Hafif log yapıları.
- Lightweight data stream işlemleri.

2. Doubly Linked List — Ne Zaman Kullanılır?

Doubly Linked List, hem ileri hem geri yönde gezinmeyi sağlayan çok yönlü bir listedir. Undo/redo, navigation veya LRU cache gibi çift yönlü dolaşma gerektiren durumlarda idealdir.

Kullanım Senaryoları

- Undo / Redo mekanizmaları (geri – ileri geçiş).
- Browser History (Back/Forward navigation).
- LRU Cache implementasyonları.
- Playlist yapıları (önceki/sonraki öge geçişi).
- UI veya veri yapılarında çift yönlü traversal gereken durumlar.

Gerçek Hayat Örnekleri

- VSCode / Photoshop / Figma undo-redo zinciri.
- Chrome / Firefox tarayıcı geçmişi.
- React Fiber (node $\rightarrow$ sibling $\rightarrow$ return pointer yapısı).
- Linux process/task scheduler listeleri.

3. Circular Linked List — Ne Zaman Kullanılır?

Circular Linked List, listenin son elemanının başlangıç elemanına işaret ettiği döngüsel bir yapıdır. "Başa dönme" ihtiyacının doğal olduğu durumlarda tercih edilir.

Kullanım Senaryoları

- Round Robin Scheduler (CPU veya thread schedule).
- Sonsuz döngü playlist yapıları.
- Oyunlarda oyuncu sırası (turn-based cycle).
- Circular buffer implementasyonları (audio/video processing).
- Ring topolojisi kullanan ağ yapıları.

Gerçek Hayat Örnekleri

- Process round-robin algoritmaları.
- Multiplayer board game oyuncu döngüsü.
- Token Ring Network.
- Ses işleme (audio circular buffer).

Tree Veri Yapıları

1. Tree Nedir?

Tree, düğümlerden (nodes) ve bu düğümler arasındaki ebeveyn-çocuk (parent-child) ilişkilerinden oluşan hiyerarşik bir veri yapısıdır. Cycle içermez ve tek bir kök (root) düğümden başlar.

Gerçek Hayat Kullanımları

- HTML/DOM Tree (frontend dünyasının temel modeli)
- Angular/React Component Tree
- Dosya sistemleri (folders/files)
- Compiler Abstract Syntax Tree (AST)
- Oyun karar ağaçları (game decision tree)
- Yapay zeka karar ağaçları

2. Binary Tree

Binary Tree, her düğümün en fazla iki çocuk (**left** ve **right**) sahip olabildiği özel bir Tree yapısıdır. Algoritmaların büyük kısmı (divide-and-conquer, traversal, recursion) Binary Tree temellidir.

3. Binary Tree Türleri

Full Binary Tree

Her düğümün ya **0** çocuğu vardır ya da **2** çocuğu vardır.

Perfect Binary Tree

Tüm seviyeler tamamen doludur. Yaprak düğümlerin (leaf nodes) tamamı aynı seviyededir.

Complete Binary Tree

Son seviye hariç tüm seviyeler doludur. Son seviye soldan sağa doğru doldurulur. Heap veri yapılarının temelidir.

Binary Search Tree (BST)

Binary Tree'nin arama için optimize edilmiş halidir. Bu sayede arama, ekleme ve silme işlemleri $O(\log n)$ ortalama karmaşıklığa sahiptir.

Heap (Priority Queue)

Tanım

Heap, elemanları belirli bir öncelik kuralına göre düzenleyen bir **priority queue** veri yapısıdır. İki temel türü vardır:

- **Min-Heap:** En küçük eleman kök (*root*) düğümdedir.
- **Max-Heap:** En büyük eleman kök düğümdedir.

Heap bir Binary Tree gibi görünse de gerçekte **array tabanlı** bir yapıdır ve *Complete Binary Tree* özelliğini taşır.

$$parent = i, \quad leftchild = 2i + 1, \quad rightchild = 2i + 2$$

Amacı

Heap'in temel amacı, en öncelikli elemanı $O(1)$ **zamanda çekebilmek** ve yeni eleman eklerken veya çıkarırken $O(\log n)$ **zamanda yeniden düzenleme** yapmaktır.

Nasıl Çalışır?

1. Ekleme (Insert) — Bubble Up

Yeni eleman dizinin sonuna eklenir ve değeri parent'ından daha önemliyse (yani Max-Heap için daha büyükse, Min-Heap için daha küçükse) yukarı doğru hareket eder:

$$swap(child, parent) \rightarrow bubble - up$$

2. Çıkarma (Pop) — Bubble Down

Kök eleman çıkarılır, dizinin sonundaki eleman köke taşınır ve doğru pozisyonu bulana kadar aşağı doğru hareket eder:

$$swap(parent, larger/smallerchild) \rightarrow bubble - down$$

Zaman Karmaşıklığı

İşlem	Süre
En öncelikli elemanı almak	$O(1)$
Ekleme (Insert)	$O(\log n)$
Silme (Pop)	$O(\log n)$
Arama (Search)	$O(n)$

Neden BST Değil?

Heap, BST gibi tam sıralı bir yapı değildir. Amaç, tüm elemanlar üzerinde sıralama sağlamak değil; **yalnızca en önemli (en büyük/en küçük) elemanı tepeye getirmektir**. Bu nedenle Heap, BST'den farklıdır ve daha hızlıdır.

Gerçek Hayat Kullanım Alanları

1. CPU / OS İşlem Zamanlayıcıları (Scheduling)

İşletim sistemi thread ve process önceliklerini Max-Heap ile yönetir.

2. Dijkstra Algoritması

Her adımda en kısa mesafeli node, Min-Heap üzerinden alınır.

3. Zamanlanmış Görevler (Timers)

JavaScript, tarayıcı motorları ve Node.js, gerçekleştirme sırası gelen işleri priority queue ile yönetir.

4. Oyun Motorları

Render, animation ve fiziksel olaylar için öncelikli event listeleri.

5. Top-K Problemleri

- En büyük 10 kullanıcı
- En küçük 5 fiyat
- En popüler 20 ürün

Kısa Özet

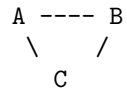
- Heap bir sıralama yapısı değildir, **öncelik yapısıdır**.
- En önemli elemanı çekmek $O(1)$ 'dir.
- Insert ve pop işlemleri $O(\log n)$ maliyetlidir.
- Scheduling, yol bulma, zamanlama ve iş kuyruğu sistemlerinde kullanılır.

Heap = VIP Kuyruğu: En öncelikli eleman her zaman en önde.

Graph (Temiz ve Düzenli Özet)

1) Graph Nedir?

Graph = Noktalar (nodes) + bağlantılar (edges) yapısıdır.



- Node = varlık
- Edge = varlıklar arası ilişki

Tree'nin daha genel hâlidir:

- Tree $\rightarrow$ her node'un tek parent'ı vardır.
- Graph $\rightarrow$ bir node istediği kadar bağlantıya sahip olabilir (cycle olabilir).

Graph, gerçek dünyayı modellemek için en uygun veri yapısıdır.

2) Graph Türleri

2.1 Undirected Graph (Yönsüz)

A ---- B

Bağlantı iki yönlüdür ($A \rightarrow B = B \rightarrow A$).

Kullanım:

- arkadaşlık grafiği
- flight connections
- network topolojisi

2.2 Directed Graph (Yönlü)

A ---> B

Tek yönlü ilişki.

Kullanım:

- Instagram “following”
- microservice call graph
- pipeline DAG

2.3 Weighted Graph (Ağırlıklı)

A --5--> B

Kenarların ağırlıkları vardır (mesafe, maliyet, zaman).

Kullanım:

- navigasyon
- teslimat rotası
- paket routing

2.4 Directed Acyclic Graph (DAG)

Döngü içermez.

A → B → D

↘ C →

Kullanım:

- Webpack dependency graph
- Nx workspace graph
- Airflow DAG
- Git commit graph

3) Graph Traversal — BFS ve DFS

Traversal = bir grafi dolaşma yöntemidir.

3.1 BFS (Breadth-First Search)

BFS, “**en yakındaki yerlere önce bak, uzaktakilere sonra bak**” mantığıyla çalışan bir graph arama algoritmasıdır.

Graph üzerindeki noktaları (node’ları) **halka halka** dolaşır:

- 1 adım uzaklıktakileri önce ziyaret eder,
- 2 adım uzaklıktakilere ancak 1. halkayı bitirdikten sonra geçer,
- 3. halkaya, 2. halka tamamen incelenmeden geçmez.

Bu nedenle unweighted (ağırlıksız) graph’ta **en kısa adım sayılı yolu her zaman BFS bulur** ve katman katman genişleyerek ilerler.

Seviye 0: A

Seviye 1: B, C

Seviye 2: D, E

Unweighted graph için en kısa adım sayısını garanti eder.

Kullanım:

- en yakın arkadaş önerisi
- 2 hop uzak kullanıcı
- path existence check
- level-order traversal

Halka Mantığı (Sezgisel Açıklama)

Başlangıç noktası A olsun.

A

B

C

B ve C → **1. halka (1 adım)**

A

B D

C E

D ve E → **2. halka (2 adım)**

A

B D F

C E

F → **3. halka (3 adım)**

Neden En Kısa Yol?

BFS'nin kuralı:

- 1 adım uzaklıktakilerin hepsine bakmadan 2. adıma geçmez.**
- 2. adımı bitirmeden 3. adıma geçmez.**

Bu nedenle A'dan F'ye giderken F'yi ilk gördüğün uzaklık, F'ye ulaşmanın **minimum adım sayısıdır**.

Gerçek Hayat Kullanımları

- Sosyal medya: 1. derece ve 2. derece arkadaş bulma
- Oyunlar: Labirentte en kısa yol bulma
- Ağ sistemleri: En az “hop” sayısı ile routing
- UI graph traversal: komponent ağacını katman katman gezmek

Tek Cümlelik Özet

BFS = En yakından başla → dışa doğru genişleyerek ara.

3.2 DFS (Depth-First Search)

Dibe in $\rightarrow$ geri çık $\rightarrow$ diğer dala git.

A
 B
 D
 C

Kullanım:

- cycle detection
- dependency çözme
- maze search

Topological Sort

Topological Sort, “**hangi işi önce yapmam gerekiyor?**” sorusuna cevap veren bir sıralama algoritmasıdır. Sadece **DAG (Directed Acyclic Graph)** üzerinde çalışır.

Temel Mantık

- Bazı işler diğerlerinin yapılmasına *bağımlıdır*.
- Önkoşulu olmayan işler önce yapılır.
- Bir iş tamamlandığında, ona bağlı olan işler “serbest” kalır.

Örnek

A yapılmadan B yapılamaz.
A yapılmadan C yapılamaz.
B ve C yapılmadan D yapılamaz.

Sıralama:

A → B → D

A → C → D

Dijkstra Algoritması

Dijkstra, **ağırlıklı bir graf üzerinde** en düşük maliyetli (en kısa toplam) yolu bulmak için kullanılır.

Temel Mantık

Bir şehirden başka şehirlere giderken tabelalara bakarak en ucuz rotayı bulmaya benzer.

Örnek

A $\rightarrow$ B (5 km)

A $\rightarrow$ C (10 km)

B $\rightarrow$ D (3 km)

C $\rightarrow$ D (2 km)

Hedef: A'dan D'ye en kısa yol.

- A $\rightarrow$ B = 5 km
- A $\rightarrow$ C = 10 km $\rightarrow$ Önce B'ye bakılır (daha yakın)

B üzerinden D'ye gidersek:

$$A \rightarrow B \rightarrow D = 5 + 3 = 8 \text{ km}$$

C üzerinden gidersek:

$$A \rightarrow C \rightarrow D = 10 + 2 = 12 \text{ km}$$

En kısa yol = **8 km**.

Design Patterns

Bu dokümanda klasik GoF (Gang of Four) tasarım patternleri Frontend perspektifiyle açıklanmış, her biri için uzun ve gerçekçi kullanım örnekleri verilmiştir.

Tüm patternler üç başlık altında toplanır:

- **Creational Patterns** (Oluşturucu)
- **Structural Patterns** (Yapısal)
- **Behavioral Patterns** (Davranışsal)

Aşağıdaki bölümlerde her pattern için:

- Ne işe yarar?
- Angular dünyasında karşılığı
- Uzun ve gerçek kod örneği

1. Singleton Pattern

Ne İşe Yarar?

Bir sınıfın yalnızca *tek bir instance* oluşturulmasını sağlar.

Angular'da Kullanımı

Angular servisleri varsayılan olarak Singleton'dır (`providedIn: 'root'`).

Angular Örneği

```
@Injectable({ providedIn: 'root' })
export class AuthService {

  private currentUser$ = new BehaviorSubject<User | null>(null);

  constructor(private http: HttpClient) {}

  login(email: string, password: string) {
    return this.http
      .post<User>('/api/login', { email, password })
      .pipe(
        tap(user => this.currentUser$.next(user))
      );
  }

  logout() {
    this.currentUser$.next(null);
  }

  get userChanges$() {
    return this.currentUser$.asObservable();
  }
}
```

2. Factory Method Pattern

Ne İşe Yarar?

Nesne oluşturma işlemini alt sınıflara bırakır.

Angular'da Kullanımı

Logger, notification, storage veya platform bazlı production/test objeleri üretmek.

Angular Örneği

```

export abstract class MessageService {
  abstract send(msg: string): void;
}

@Injectable()
export class ConsoleMessageService extends MessageService {
  send(msg: string) {
    console.log('Console:', msg);
  }
}

@Injectable()
export class AlertMessageService extends MessageService {
  send(msg: string) {
    alert('Alert: ' + msg);
  }
}

/* --- 2) Factory --- */

@Injectable({ providedIn: 'root' })
export class MessageFactory {
  constructor(
    private consoleImpl: ConsoleMessageService,
    private alertImpl: AlertMessageService
  ) {}

  create(type: 'console' | 'alert'): MessageService {
    return type === 'console'
      ? this.consoleImpl
      : this.alertImpl;
  }
}

/* --- 3) Component içinde kullanım --- */

@Component({
  selector: 'app-example',
  template: `<button (click)="notify()">Notify</button>`
})
export class ExampleComponent {
  constructor(private factory: MessageFactory) {}

  notify() {
    const messenger = this.factory.create('alert');
    messenger.send('Hello Angular!');
  }
}

```

3. Abstract Factory Pattern

Ne İşe Yarar?

Birbiriyle ilişkili nesneleri tutarlı bir şekilde oluşturmak.

Angular'da Kullanımı

Tema, UI bileşen seti, platform (web/mobile) bağımlı component/service üretimi.

Angular Örneği

```

export abstract class NotificationService {
  abstract notify(message: string): void;
}

/* Implementations */
@Injectable()
export class SnackbarNotificationService implements NotificationService {
  constructor(private snack: MatSnackBar) {}
  notify(message: string) {
    this.snack.open(message, 'OK', { duration: 3000 });
  }
}

@Injectable()
export class ToastNotificationService implements NotificationService {
  constructor(private toastr: ToastrService) {}
  notify(message: string) {
    this.toastr.success(message);
  }
}

/* Injection Token */
export const NOTIFICATION = new InjectionToken<NotificationService>('NOTIFICATION');

/* Factory */
export function notificationFactory(
  isMaterial: boolean,
  snack: MatSnackBar,
  toastr: ToastrService
): NotificationService {
  return isMaterial
    ? new SnackbarNotificationService(snack)
    : new ToastNotificationService(toastr);
}

/* Provider */
@NgModule({
  providers: [
    {
      provide: NOTIFICATION,
      useFactory: notificationFactory,
      deps: ['IS_MATERIAL_DESIGN', MatSnackBar, ToastrService]
    }
  ]
})
export class CoreModule {}

```

Factory Method vs Abstract Factory

1. Kısa Tanımlar

- **Factory Method:** Tek bir ürünün farklı türlerini seçip üreten bir yöntemdir. Ürünün nasıl oluşturulacağı alt sınıflara bırakılır.
- **Abstract Factory:** Birbirleriyle ilişkili ürünlerin tamamını (ürün ailesi) uyumlu şekilde üreten fabrika modelidir.

2. Temel Fark

- **Factory Method:** Bir *ürün türü* seçilir. (Örn: Snackbar mı Toastr mı?)
- **Abstract Factory:** Bir *ürün ailesi* seçilir. (Örn: Material UI seti mi, Bootstrap UI seti mi?)

Angular Özelinde En Pratik Fark

Factory Method

- Sadece **tek bir servis** seçilir.
- Örnek: "SnackBar mı Toastr mı?"

```
factory.create('snackbar');
```

Abstract Factory

- Tema veya platforma göre **komple bir servis/component seti** seçilir.
- Örnek: Material UI bileşen ailesi üretmek.

```
MaterialUIFactory.createButton()  
MaterialUIFactory.createInput()  
MaterialUIFactory.createCard()
```

Hepsi birlikte değişir.

4. Builder Pattern

Ne İşe Yarar?

Karmaşık nesneleri adım adım oluşturmayı sağlar.

Angular'da Kullanımı

Reactive Form Builder, Query Builder, HTTP param builder.

Angular Örneği

```
export class UserQueryBuilder {
  private params: any = {};

  byName(name: string) {
    this.params.name = name;
    return this;
  }

  byAge(min: number, max: number) {
    this.params.minAge = min;
    this.params.maxAge = max;
    return this;
  }

  sortBy(field: string) {
    this.params.sort = field;
    return this;
  }

  build() {
    return this.params;
  }
}

// Kullanım
const query = new UserQueryBuilder()
  .byName('Ömer')
  .byAge(20, 40)
  .sortBy('createdAt')
  .build();

this.http.get('/api/users', { params: query });
```

5. Adapter Pattern

Ne İşe Yarar?

Uyumsuz arayüzleri birbirine dönüştürür.

Angular'da Kullanımı

API DTO → UI model çevirisi (en çok kullanılan patternlerden biridir).

Angular Örneği

```
interface UserDTO {
  id: number;
  full_name: string;
  mail: string;
}

interface User {
  id: number;
  name: string;
  email: string;
}

@Injectable({ providedIn: 'root' })
export class UserAdapter {
  static fromDTO(dto: UserDTO): User {
    return {
      id: dto.id,
      name: dto.full_name,
      email: dto.mail
    };
  }
}

@Injectable({ providedIn: 'root' })
export class UserService {
  constructor(private http: HttpClient) {}

  getUsers() {
    return this.http.get<UserDTO[]>('/api/users')
      .pipe(map(list => list.map(UserAdapter.fromDTO)));
  }
}
```

6. Strategy Pattern

Ne İşe Yarar?

Değiştirilebilir algoritmalar sunar.

Angular'da Kullanımı

Sorting, filtering, validation stratejileri.

Angular Örneği

```
export interface SortStrategy {
  sort(list: any[]): any[];
}

export class AscSort implements SortStrategy {
  sort(list: any[]) {
    return [...list].sort();
  }
}

export class DescSort implements SortStrategy {
  sort(list: any[]) {
    return [...list].sort().reverse();
  }
}

@Injectable({ providedIn: 'root' })
export class SortContext {
  constructor(private strategy: SortStrategy) {}

  sort(items: any[]) {
    return this.strategy.sort(items);
  }
}
```

7. Facade Pattern

Ne İşe Yarar?

Karmaşık sistemi basit bir yüz ile sunar.

Angular'da Kullanımı

AuthFacade, UserFacade, ProductFacade.

Angular Örneği

Sorun (Facade Olmadan)

Bir component aynı anda birden fazla servisten veri dinlemek istediğinde genellikle birden çok `subscribe` çağrısı yapmak zorunda kalır:

```
this.userService.user$.subscribe(...);  
this.ordersService.orders$.subscribe(...);  
this.notificationsService.unreadCount$.subscribe(...);
```

Bu yaklaşım:

- birden fazla subscribe yönetimi,
- ayrı ayrı error handling,
- ayrı ayrı loading state,
- component kodunun şişmesi

gibi sorunlara yol açar.

Çözüm (Facade ile Tek Subscribe)

Facade, birden fazla kaynağı **tek bir observable state** içine toplar. Component ise bu state'i sadece **bir kez** dinler.

```
@Injectable({ providedIn: 'root' })
export class UserDashboardFacade {

  readonly vm$ = combineLatest({
    user: this.userService.user$,
    orders: this.ordersService.orders$,
    unreadCount: this.notificationsService.unreadCount$
  });

  constructor(
    private userService: UserService,
    private ordersService: OrdersService,
    private notificationsService: NotificationsService
  ) {}

  loadDashboardData(userId: string) {
    this.userService.loadUser(userId);
    this.ordersService.loadOrders(userId);
    this.notificationsService.refresh();
  }

  markAllAsRead() {
    this.notificationsService.markAllAsRead();
  }
}
```

Component Tarafı (Tek Async Pipe = Tek Subscribe)

```
@Component({
  selector: 'app-user-dashboard',
  template: `
    <ng-container *ngIf="vm$ | async as vm">
      <h1>{{ vm.user.name }}</h1>
      <p>Unread: {{ vm.unreadCount }}</p>

      <ul>
        <li *ngFor="let order of vm.orders">
          {{ order.title }}
        </li>
      </ul>
    </ng-container>
  `
})
export class UserDashboardComponent {
  vm$ = this.facade.vm$;

  constructor(private facade: UserDashboardFacade) {
    this.facade.loadDashboardData('123');
  }
}
```

Açıklama

Facade burada şu işi yapar:

- user, orders ve unreadCount akışlarını tek bir **vm\$** observable'ında birleştirir,
- component yalnızca **bir tane async pipe** ile tüm state'i alır,
- böylece UI tarafında tek subscribe yapılmış olur.

Özet: Normalde 3 ayrı subscribe gerektiren işi Facade tek bir subscribe'a indirger. Component sadeleşir, UI state yönetimi temizlenir.

8. Observer Pattern

Ne İşe Yarar?

Bir nesnede değişiklik olduğunda tüm subscriber'lara bildirim gönderilir.

Angular'da Kullanımı

RxJS Subject & BehaviorSubject.

Angular Örneği

```
@Injectable({ providedIn: 'root' })
export class NotificationsService {
  private subject = new Subject<string>();
  notifications$ = this.subject.asObservable();

  notify(msg: string) {
    this.subject.next(msg);
  }
}

// Component
this.notificationsService.notifications$.subscribe(msg => {
  console.log('Notification received:', msg);
});
```

Proxy Pattern

Amaç

Bir işlemin önüne “aracı” koyarak isteği kontrol etmek: cache, retry, logging, auth eklemek.

Angular Örneği (Interceptor = Proxy)

```
@Injectable()
export class AuthProxyInterceptor implements HttpInterceptor {
  intercept(req: HttpRequest<any>, next: HttpHandler) {
    const clone = req.clone({
      setHeaders: { Authorization: 'Bearer token123' }
    });
    return next.handle(clone);
  }
}
```

Özet

- HttpInterceptor = Proxy Pattern.
- “İstek gitmeden önce araya gir → davranışı değiştir.”

Decorator Pattern

Amaç

Bir sınıfa, orijinal kodunu değiştirmeden ek özellik kazandırmak.

Angular Örneği

```
/* Decorator: Metot çalışmadan önce loading açar,
   bittikten sonra kapatır. */
function AutoLoading(loaderKey: keyof any) {
  return function(target: any, key: string, desc: PropertyDescriptor) {
    const original = desc.value;

    desc.value = async function(...args: any[]) {
      this[loaderKey] = true;
      try {
        return await original.apply(this, args);
      } finally {
        this[loaderKey] = false;
      }
    };
  };
}

@Injectable({ providedIn: 'root' })
export class UserService {

  loading = false;

  @AutoLoading('loading')
  async loadUser(id: string) {
    // fake API
    await new Promise(res => setTimeout(res, 1000));
    return { id, name: 'Ömer' };
  }
}
```

Özet

- Metot çalışınca otomatik olarak loading = true olur.
- Metot bitince otomatik olarak loading = false olur.
- Kod tekrarını azaltır, UI state'i sadeleştirir.
- Metoda ek davranış ekler → Decorator Pattern'ın FE karşılığı.
- Angular @Component, @Injectable dekoratörleri de birer decorator'dır.

Command Pattern

Amaç

Her işlemi bir “komut” nesnesi haline getirip undo/redo yapabilmek.

Angular Örneği

```
export interface Command {
  execute(): void;
  undo(): void;
}

export class UpdateNameCommand implements Command {
  constructor(private form: FormGroup, private value: string) {}

  private oldValue = this.form.value.name;

  execute() { this.form.patchValue({ name: this.value }); }
  undo()     { this.form.patchValue({ name: this.oldValue }); }
}

@Injectable({ providedIn: 'root' })
export class CommandService {
  private history: Command[] = [];

  run(cmd: Command) {
    cmd.execute();
    this.history.push(cmd);
  }

  undo() {
    const cmd = this.history.pop();
    cmd?.undo();
  }
}
```

State Pattern

Amaç

Component davranışının state'e göre otomatik değişmesi.

```
type LoginState = 'idle' | 'loading' | 'success' | 'error';

@Injectable({ providedIn: 'root' })
export class LoginStateService {
  private stateSubject = new BehaviorSubject<LoginState>('idle');
  state$ = this.stateSubject.asObservable();

  setState(state: LoginState) {
    this.stateSubject.next(state);
  }
}

@Component({
  selector: 'app-login',
  template: `
    <button *ngIf="state==='idle'" (click)="login()">Login</button>
    <p *ngIf="state==='loading'">Loading...</p>
    <p *ngIf="state==='success'">Welcome!</p>
    <p *ngIf="state==='error'">Failed.</p>
  `
})
export class LoginComponent {
  state: LoginState = 'idle';

  constructor(private s: LoginStateService) {
    this.s.state$.subscribe(st => this.state = st);
  }

  login() {
    this.s.setState('loading');
    setTimeout(() => this.s.setState('success'), 1000);
  }
}
```

Özet

- Durum değiştikçe ekran otomatik değişir.
- “State = behavior” mantığı.

Chain of Responsibility

Amaç

Bir isteğin birden fazla aşamadan (interceptor zinciri) geçmesi.

```
@Injectable()
export class AuthInterceptor implements HttpInterceptor {
  intercept(req, next) {
    const cloned = req.clone({ setHeaders: { Authorization: 'Token' }});
    return next.handle(cloned);
  }
}

@Injectable()
export class LoggingInterceptor implements HttpInterceptor {
  intercept(req, next) {
    console.log('Request:', req.url);
    return next.handle(req);
  }
}
```

Bir HTTP isteği, işlem hattı (pipeline) boyunca **sırayla** birden fazla **interceptor** içinden geçer. Her interceptor isteği:

- değiştirebilir,
- loglayabilir,
- doğrulayabilir,
- durdurabilir,
- olduğu gibi devam ettirebilir.

Her interceptor, işini bitirdikten sonra isteği bir sonrakine iletir:

$$Request \rightarrow I_1 \rightarrow I_2 \rightarrow I_3 \rightarrow Server$$

Bu yapı **Chain of Responsibility Pattern**'ın birebir karşılığıdır.

Interceptor Zincirinin Akışı

$$HttpRequest \rightarrow AuthInterceptor \rightarrow LoggingInterceptor \rightarrow Backend$$

Her interceptor bir “halkadır”. Hiçbiri isteği sonuçlandırmaz — sadece işler ve sıradakine aktarır.

Açıklama

- **AuthInterceptor** → isteğe token ekler.
- **LoggingInterceptor** → isteğin URL'ini loglar.
- Her biri kendi görevini yaptıktan sonra isteği bir sonrakine iletir.

Bu nedenle Angular HttpInterceptor mekanizması, **Chain of Responsibility Pattern**'ın tam bir uygulamasıdır.

Template Method

Template Method Pattern, bir işlemin **iskeletini (adım sırasını)** üst sınıfta sabitleyen; ancak bu adımların **içeriklerinin** alt sınıflar tarafından tanımlanmasına izin veren tasarım desenidir. Bir işlem her zaman aynı düzenle ilerler; fakat adımların *nasıl* uygulanacağı alt sınıflar tarafından belirlenir.

- İşlem sırası = değişmez (üst sınıfta tanımlıdır).
- Her adımın detayı = değişebilir (alt sınıf override eder).

Günlük Hayattan Örnek

Kek yapma sürecinin sırası değişmez:

1. Malzemeleri hazırla
2. Karıştır
3. Pişir
4. Servis et

Ancak kekin türü değişebilir: çikolatalı, muzlu, sade vb.
Sıra aynı kalır; değişen şey sadece adımların içeriğidir.

```
export abstract class BaseForm {
  submit() {
    this.validate();
    this.doSubmit();
    this.afterSubmit();
  }

  abstract validate(): void;
  abstract doSubmit(): void;

  afterSubmit() {
    console.log('Done');
  }
}

export class UserForm extends BaseForm {
  validate() { console.log('User validated'); }
  doSubmit() { console.log('User submitted'); }
}
```

Özet

- İş akışı sabit.
- Bazı adımlar subclass tarafından doldurulur.

Composite Pattern

Composite Pattern, iç içe geçmiş öğelerden oluşan bir yapıyı hem tek bir öğeymiş gibi hem de öğeler koleksiyonuymuş gibi aynı arabirimle işleyebilmeni sağlayan yapıdır.

```
export interface MenuItem {
  label: string;
  children?: MenuItem[];
}

@Component({
  selector: 'app-menu',
  template: `
    <ul>
      <li *ngFor="let item of items">
        {{ item.label }}
        <app-menu *ngIf="item.children" [items]="item.children"></app-menu>
      </li>
    </ul>
  `
})
export class MenuComponent {
  @Input() items: MenuItem[] = [];
}
```

Özet

- “Leaf” ve “Composite” aynı arayüzü kullanır.
- Angular’da recursive component tam Composite’tir.

Memento Pattern

Amaç

Bir formun önceki halini kaydetmek (undo yapabilmek).

```
@Injectable({ providedIn: 'root' })
export class FormHistoryService {
  private history: any[] = [];

  save(state: any) {
    this.history.push(JSON.parse(JSON.stringify(state)));
  }

  undo() {
    return this.history.pop();
  }
}
```

Özet

- Form snapshot'ı = Memento.
- Undo/redo mekanizmalarının temelidir.

Interpreter Pattern

Amaç

Interpreter Pattern, metinsel bir kuralı veya sorgu ifadesini ("age > 18", "status:active", "price < 100") program tarafından çalıştırılabilir bir yapıya dönüştürme işlemidir.

Basit Açıklama

Bir string kural alınır ve anlamı **yorumlanarak** gerçek bir işleme dönüştürülür:

" > 10" → *value* > 10, "status : active" → *user.status* = "active"

Frontend Kullanımları

- Admin panellerinde arama / filtre DSL yorumlama
- is:open, tag:bug gibi mini sorguları parse etme
- API'den gelen validation kurallarını ("min:10") yorumlama
- Dinamik filtreleme/sıralama kurallarını string olarak işleme

```
@Injectable({ providedIn: 'root' })
export class FilterInterpreter {
  interpret(expr: string, value: number) {
    if (expr === ">10") return value > 10;
    if (expr === "<5") return value < 5;
    return false;
  }
}
```

Özet

- Mini kural dilleri için kullanılır.

Flyweight Pattern

Tanım

Flyweight Pattern, aynı türden çok sayıda nesnenin oluşturulduğu durumlarda, bu nesnelerin **ortak olan kısımlarını tek bir yerde** tutup, sadece değişen küçük bilgileri ayrı nesnelerle temsil ederek hafıza kullanımını büyük ölçüde azaltan bir yapıdır.

Frontend Açıklaması

Frontend dünyasında Flyweight Pattern'in en yaygın karşılığı:

- büyük listelerde (10.000+ öğe),
- tüm elemanları DOM'da oluşturmadan,
- sadece ekranda görünen küçük bir kısmını render ederek,
- geri kalanını hafif temsilcilerle (flyweight objects) yönetmektir.

Bu yaklaşım sayesinde: 10000 öğeden 20 gerçek DOM elemanı

Angular'da Gerçek Karşılığı

Angular CDK Virtual Scroll mekanizması Flyweight Pattern'in birebir örneğidir.

- DOM'da yalnızca görünür eleman sayısı kadar component bulunur.
- Kullanıcı scroll ettikçe aynı component instance'ları yeniden kullanılır.
- Yeni DOM yaratılmadan yalnızca içerik güncellenir.

Bu, Flyweight'in temel fikridir: **“Paylaşılabilir olan tek bir nesne, binlerce öğeye hizmet eder.”**

Örnek Kod

```
@Component({
  selector: 'app-big-list',
  template: `
    <cdk-virtual-scroll-viewport itemSize="40" style="height:300px">
      <div *cdkVirtualFor="let item of items">
        {{ item }}
      </div>
    </cdk-virtual-scroll-viewport>
  `
})
export class BigListComponent {
  items = Array.from({ length: 10000 }).map((_, i) => `Item ${i}`);
}
```

Özet

- Flyweight → hafif nesneler + ortak yapı.
- Amaç → çok sayıda öğeyi performanslı şekilde işlemek.
- Angular'daki en doğal örnek → **CDK Virtual Scroll**.
- Kısa tanım → *“10.000 öğe var ama DOM'da 20 tane var.”*

SOLID Prensipleri (FE Developer Perspektifi)

Frontend geliştiriciler için SOLID prensipleri; **component mimarisi**, **state yönetimi**, **UI katmanı**, **service yapıları** ve ;abstraction layer'ları daha sürdürülebilir ve değişime açık hale getirmek için kullanılır.

Aşağıdaki başlıklar tamamen **React**, **Angular** ve benzeri UI mimarilerine göre düzenlenmiştir.

S — Single Responsibility Principle

Tanım: Her component, hook, service veya fonksiyon yalnızca *tek bir sorumluluğa* sahip olmalıdır.

Frontend'de Karşılığı

- Bir component hem UI render edip hem data-fetch etmemelidir.
- Bir service hem API hem global state güncelleyici olmamalıdır.
- Bir fonksiyon hem validation hem formatting yapmamalıdır.

Yanlış Kullanım (Anti-Pattern)

```
// React | UI + Fetch aynı yerde
function UserCard() {
  const [user, setUser] = useState(null);
  useEffect(() => {
    fetch('/api/user').then(r => r.json()).then(setUser);
  }, []);
  return <div>{user.name}</div>;
}
```

Doğru Kullanım

```
// Data fetching ayrı
const useUser = () => { ... };

// UI ayrı
function UserCard() {
  const user = useUser();
  return <UIView user={user} />;
}
```

O — Open/Closed Principle

Tanım: Kod genişlemeye açık, fakat değiştirilmeye kapalı olmalıdır.

Frontend’de Karşılığı

- Component’in içine onlarca if/else eklemek yerine prop veya slot ile genişletilebilir yapmak.
- Form validation kurallarının component içinde hard-coded olmaması.

Yanlış Kullanım

```
function Button({ type }) {  
  if (type === 'primary') return <button class="p">...</button>  
  if (type === 'danger') return <button class="d">...</button>  
}
```

Doğru Kullanım

```
function Button({ className, ...rest }) {  
  return <button className={className} {...rest} />  
}
```

L — Liskov Substitution Principle

Tanım: Aynı interface’i sağlayan her şey birbirinin **yerine geçebilir** olmalıdır.

Frontend’de Karşılığı

- UI, gerçek API yerine mock API kullanılınca bozulmamalıdır.
- Form validator değişince form patlamamalıdır.
- Strategy pattern ile sorting algoritması değişince UI aynı çalışmalıdır.

Yanlış Kullanım

```
function sort(items) {  
  if (!items) return null; // tüketen kodu kırar  
  return items.sort();  
}
```

Doğru Kullanım

```
interface SortStrategy { sort(items: any[]): any[]; }  
  
class AlphabeticSort implements SortStrategy {  
  sort(items) { return [...items].sort(); }  
}
```

React Native ve React İçin Storage Tipleri ve Popüler Kütüphaneler

Aşağıdaki tablo ve listeler, modern React Native ve React uygulamalarında kullanılan tüm depolama türlerini ve her tür için en yaygın kütüphaneleri özetlemektedir.

1. Key–Value Storage (Basit ayarlar, token, küçük veri)

React Native

- `react-native-mmkv` — En hızlı C++ tabanlı key-value storage
- `@react-native-async-storage/async-storage` — Resmi AsyncStorage alternatifi
- `expo-mmkv` — Expo için MMKV wrapper

React (Web)

- `localStorage`, `sessionStorage`
- `zustand persist`, `jotai-storage`

2. Secure Storage (Güvenli, şifreli depolama)

React Native

- `react-native-keychain` — OS-level secure store
- `react-native-encrypted-storage` — AES şifreli storage

React (Web)

- HttpOnly cookies (JS erişemez)
- Web Crypto API (`window.crypto.subtle`)

3. In-Memory Store (RAM üzerinde, geçici state)

React Native / React

- Zustand, Jotai, Recoil, MobX, Redux Toolkit
- TanStack Query cache
- TanStack DB (reactive mini database)
- XState (state machine/flow)

4. SQL & Database Storage (Offline-first, büyük veri)

React Native

- `react-native-sqlite-storage`, `expo-sqlite`
- WatermelonDB (sync-first, performanslı)
- Realm DB (NoSQL, offline-first)
- TypeORM + RN driver

React (Web)

- IndexedDB (Dexie.js)
- Lovefield (SQL-like)
- RxDB, PouchDB (sync-first)

5. File Storage (Fotoğraf, video, pdf, blob)

React Native

- `react-native-fs`
- `expo-file-system`
- `react-native-fast-image` (cache)

React (Web)

- File System Access API
- Cache API (Service Worker)
- IndexedDB (blob desteği ile)

6. Cloud Storage (Uzaktan dosya depolama)

- Firebase Storage
- AWS S3 (`aws-amplify/storage`)
- Supabase Storage
- Appwrite Storage

7. Cache Storage (API sonuçları için)

React Native / React

- TanStack Query (data fetching cache)
- Axios cache adapter
- SWR
- AsyncStorage/MMKV persist

8. Sync-First / Offline-First Storage

React Native

- WatermelonDB — offline-first + sync engine
- Realm DB — live objects + sync desteği
- PouchDB — CouchDB ile eşitleme

9. Özet Tablo

Kullanım	Önerilen Depolama
Token / Güvenli veri	Secure Storage / Keychain
Küçük ayarlar	MMKV
UI state	Zustand / Redux / NgRx / Jotai
API cache	TanStack Query
Entity data (tablo yapısı)	TanStack DB
Offline-first büyük veri	WatermelonDB / Realm
Dosya depolama	react-native-fs / expo-file-system
Bulut dosya depolama	Firebase Storage / S3 / Supabase

Unified Logging Architecture

Modern dağıtık sistemlerde, bir kullanıcının tetiklediği işlemi **frontend** → **gateway** → **backend** → **microservice** → **database** zinciri boyunca uçtan uca takip edebilmek kritik bir gereksinimdir. Bu mimariyi mümkün kılan temel bileşenler:

- Frontend logları
- Backend logları
- API Gateway logları
- Client-side error monitoring (Sentry vb.)
- Performans logları
- Trace-ID propagation
- Correlation ID

Bu bileşenler olmadan, örneğin “UI bozuldu” gibi bir hata **hangi katmanda gerçekleştiği belirsiz** hâlde kalır. Özellikle **Trace-ID** kavramını bilmek ve uygulamak zorunludur.

Trace ID

Trace ID, bir kullanıcının tetiklediği tekil bir isteğin, sistem boyunca **tek bir kimlikle işaretlenmesini** sağlayan global tanımlayıcıdır.

- Frontend, bir istek oluştuğunda bir UUID üretir.
- Bu değer HTTP header üzerinden backend’e geçer.
- API Gateway aynı Trace ID’yi loglar ve servis çağrılarına ekler.
- Backend, tüm log satırlarına aynı Trace ID’yi ekler.
- Servisler arası çağrılar, aynı Trace ID ile propagate edilir.
- Veri tabanı seviyesinde bile bu ID loglanabilir.

Örneğin:

```
traceId = 4df2a12b-e171-4e01-92a1-3610ad1c8a19
```

Bu değer sayesinde bir isteğin yaşam döngüsü uçtan uca takip edilebilir:

Frontend → Gateway → Backend → Microservice → Database

Correlation ID

Trace ID tek bir isteği temsil ederken, Correlation ID bir iş akışı kapsamında gerçekleşen **birden fazla isteği birbirine bağlamak** için kullanılır.

Örneğin bir sipariş akışı:

- Ödeme isteği (traceId_1)
- Stok rezervasyonu (traceId_2)
- Bildirim gönderme (traceId_3)

Hepsi aynı `correlationId = orderId` değerini taşır. Bu sayede “Bu siparişle ilgili tüm işlemleri getir” denebilir.

Frontend Logları

Frontend katmanı, kullanıcı davranışlarını ve tarayıcı tarafındaki hataları loglar:

- Buton tıklamaları
- Network request hataları
- JS runtime hataları
- UI rendering problemleri
- Performans metrikleri (LCP, FCP)

Sentry, Datadog veya LogRocket gibi araçlar kullanılır. Trace ID frontend’de üretilir ve backend’e taşınır.

Backend Logları

Backend, operasyonel hataların ve iş akışlarının kaynağıdır.

- Request input
- Kullanıcı kimliği
- İşlem süresi
- Hata stack trace
- Servis çağrıları
- Cache hit/miss
- SQL sorgu süreleri

Her log satırı ortak bir formatta olmalı:

```
level=info traceId=abc-123 correlationId=987 user=123 action="payment_start"
```

API Gateway Logları

API Gateway (Kong, Nginx, Istio, Traefik vb.) şu bilgileri loglar:

- IP ve User-Agent
- Endpoint ve latency
- Rate-limit sonuçları
- JWT doğrulama detayları

Gateway Trace ID'yi zorunlu olarak propagate eder.

Performance Logs

Performans logları her request'in hangi aşamada geciktiğini analiz etmeyi sağlar:

- Gateway latency
- Backend latency
- Database latency
- Harici servis çağrısı gecikmeleri

Örnek:

```
[traceId=abc-123] SQL took 780 ms (slow query)
```

Trace-ID Propagation

Trace ID, her katmanda HTTP header üzerinden taşınır:

```
X-Trace-Id: abc-123  
X-Correlation-Id: order-987
```

Backend kodu:

```
String traceId = request.getHeader("X-Trace-Id");  
MDC.put("traceId", traceId);
```

Tüm servisler bu ID'yi propagation mekanizmasıyla taşır.

Correlation ID

Dağıtık sistemlerde tek bir iş akışı, çoğu zaman birden fazla HTTP isteği ve servis çağrısından oluşur. Bu çağrıların her birinde farklı **Trace ID** bulunur; ancak işin bütünü tek bir kimlik altında toplamak için **Correlation ID** kullanılır.

Correlation ID, bir iş sürecine ait tüm request ve eventlerin ortak olarak taşıdığı üst seviyeli kimliktir. Trace ID tekil bir isteği temsil ederken, Correlation ID işin tamamını temsil eder.

- **Trace ID**: Tek bir isteğin sistemdeki yolculuğunu ifade eder.
- **Correlation ID**: Bir iş akışındaki birden fazla isteği birbirine bağlar.

Özetle:

Trace ID → mikro seviye (tek request)

Correlation ID → makro seviye (iş akışının bütünü)

Örnek: Pizza Sipariş Akışı

Bir pizza siparişi verildiğinde sistemde tek bir request gerçekleşmez. Aşağıdaki işlemler farklı servisler tarafından, farklı zamanlarda, farklı Trace ID'lerle yürütülür:

- Sipariş oluşturma
- Ödeme işlemi
- Stok düşme
- Kurye atama
- Bildirim gönderme

Her işlem kendi Trace ID'sini üretir:

```
traceId = req-1    (sipariş oluşturma)
traceId = req-2    (ödeme)
traceId = req-3    (stok)
traceId = req-4    (kurye)
traceId = req-5    (bildirim)
```

Ancak hepsi aynı Correlation ID'yi taşır:

```
correlationId = order-845712
```

Bu sayede sistem:

”Bu siparişle ilgili tüm olayları getir.”

sorgusunu tek bir kimlikle gerçekleştirebilir.

Log Örneği

Aşağıdaki loglar farklı servislerden geliyor olsa bile, Correlation ID sayesinde aynı işin parçası oldukları anlaşılır:

```
[correlationId=845712] Payment succeeded  
[correlationId=845712] Stock reserved  
[correlationId=845712] Courier assigned  
[correlationId=845712] Notification sent
```

Her satır farklı bir Trace ID'ye sahip olabilir, ancak Correlation ID ortaktır.

Neden Gereklidir?

- Bir iş akışını oluşturan tüm request'leri gruplamak.
- Sipariş, ödeme, stok veya bildirim gibi alt süreçlerin bütününe inceleyebilmek.
- Dağıtık sistemlerde tutarlılık ve gözlemlenebilirlik sağlamak.
- Sorun tespitini kolaylaştırmak:
 - Ödeme başarılı → stok başarısız mı?
 - Bildirim gönderilmedi mi?
 - Adımların zamanlamaları nerede uzadı?

API Gateway: Strategic Boundary Layer

API Gateway, modern dağıtık sistemlerde **frontend ile backend** arasında konumlanan *stratejik bir sınır katmanıdır*. Bu katman yalnızca bir yönlendirme noktası değil, aynı zamanda güvenlik, standartlaştırma, kontrol ve politika uygulama bileşenlerinin tamamının toplandığı **merkezi bir mimari katmandır**.

Gateway'in Katman Olmasını Sağlayan Temel Nedenler

Tüm Trafığın Giriş Noktası

Tüm HTTP istekleri ilk olarak API Gateway'den geçer:

Frontend → API Gateway → Microservices

Bu nedenle gateway bir *boundary layer* işlevi görür.

Tüm Sistem Politikalarının Uygulama Noktası

Gateway, sistem genelinde uygulanması gereken pek çok politikayı merkezi olarak yönetir:

- Authentication & Authorization
- Rate limiting
- CORS kontrolü
- Routing kuralları
- Header canonicalization
- Request/response logging
- Error standardization
- Canary routing
- Retry & circuit breaker mekanizmaları
- Request validation

Bu nedenle gateway, politika uygulama (policy enforcement) katmanıdır.

Backend Servis Topolojisini Gizleyen Katman

Frontend yalnızca tek bir giriş noktasını bilir:

/api/

API Gateway bu istekleri arkadaki çok sayıda servise dağıtır:

- orders-service
- users-service
- payments-service
- inventory-service
- logging-service

Frontend bu detayların hiçbirini bilmez. Gateway backend yapısını soyutlayan bir adaptasyon katmanını görevi görür.

Versiyonlama Politikasının Merkezi

UI yalnızca /v1, /v2 vb. versiyonları görür. Arkada farklı servis versiyonları, farklı deployment yapıları bulunabilir; bunların tamamı gateway üzerinden yönetilir.

Hata Formatlarının Tekilleştirilmesi

Her mikroservis farklı programlama dillerinde, farklı hata formatları üretiyor olabilir. API Gateway, bu farklılıkları ortadan kaldırarak frontend için **tek bir standardize hata formatı** üretir.

Bu, hem frontend geliştirme deneyimini sadeleştirir hem de gözlemlenebilirliği artırır.

Güvenlik Sınırı (Security Boundary)

API Gateway, dış dünyaya açılan *tek* uçtur. Gerçek mikroservislerin dışarıdan direkt erişilmesi engellenir. Bu nedenle gateway aynı zamanda bir **security layer** olarak görev yapar.

API Gateway'in Alt Katmanları

API Gateway, aslında kendi içinde çok katmanlı bir yapıdan oluşur:

Security Layer	(auth, rate limit, CORS)
Routing Layer	(service discovery, path rules)
Transformation Layer	(header/body transform)
Error Handling Layer	(unified error format)
Observability Layer	(logs, metrics, tracing)

Bu katmanların tümü birlikte çalışarak API Gateway'i **modern sistemlerin en kritik sınır katmanı** hâline getirir.

Hot-Path Optimization

Hot-path, bir frontend uygulamasında *en sık yürütülen, en çok render edilen ve performansa en fazla etki eden kritik yürütme yolunu* ifade eder. Uygulama performansının büyük kısmı (yaklaşık %80'i), bu kritik akışlarda ortaya çıkar.

Hot-Path'ın Tipik Kaynakları

- En sık render olan component'ler (ör. mesaj listesi, ürün listesi, feed)
- En sık çağrılan API'ler (ör. /me, /notifications, /feed)
- Sürekli çalışan UI akışları (scroll, input değişimleri, autocomplete)
- Çok ziyaret edilen ekranlar (home, dashboard, search)
- Yoğun state güncellemeleri (selectors, reactive akışlar)

Hot-path üzerinde yapılan mikro optimizasyonlar, uygulamanın tamamında hissedilir bir hız artışı sağlar. Hot-path bilinmiyorsa:

- UI gecikmeleri ve takılmalar oluşur,
- gereksiz re-render döngüleri artar,
- scroll ve animasyonlar “lag” üretir,
- mobilde CPU ve batarya tüketimi yükselir,
- kullanıcı genel olarak uygulamayı “yavaş” algılar.

Hot-Path Optimizasyon Teknikleri

1. **Memoization** Aynı hesaplamaların tekrar yapılmasını engelleyerek render maliyetini azaltır. (React: `useMemo`, `useCallback`; Angular: `OnPush`, pure pipes, memoized selectors.)
2. **Cache TTL (Time To Live)** En sık kullanılan veriler, tanımlı bir ömür değeri ile cache'de tutulur; gereksiz API çağrıları azaltılır.
3. **Background Refresh** UI ilk etapta cache'den hızlı yüklenir, veri arka planda sessizce güncellenir.
4. **Stale-While-Revalidate (SWR)** Kullanıcıya anında cache verilir (stale), ardından arka planda yeni veri alınarak UI güncellenir (revalidate). Modern frontend veri katmanlarının temelidir.
5. **Predictive Fetch** Kullanıcının bir sonraki eylemini tahmin ederek ilgili veriyi önceden yükler. (Örneğin kullanıcı chat listesine girince, olası açacağı ilk sohbetin mesajlarını preload etmek.)
6. **Virtual List Rendering** Binlerce item yerine yalnızca ekranda görünen küçük bir bölüm render edilir. Feed, chat, infinite-scroll gibi alanlarda zorunluluktur.

Frontend–Backend Arası Katman: Client-Side Gateway (Interceptor Layer)

Modern frontend mimarisinde, API Gateway’e ek olarak uygulama içinde **frontend ile backend arasına konumlanan yazılımsal bir ara katman** bulunur. Bu katman, backend dünyasının güvenlik ve yönlendirme kapasitesine sahip olmamakla birlikte, client tarafında request/response yönetimini standartlaştıran bir **client-side gateway** işlevi görür.

Bu yapı farklı frameworklerde şu isimlerle anılır:

- API Client Layer
- HTTP Service Layer
- Client Gateway
- Interceptor Layer (Angular’da resmi olarak)
- Axios / Fetch Interceptor
- Adapter Layer
- Network Layer
- Request Pipeline

Bu katman, frontend ile backend arasında mantıksal bir sınır oluşturarak UI’nin doğrudan backend ile konuşmasını engeller.

Client-Side Gateway’in Sorumlulukları

Aşağıdaki işlevler client-side gateway katmanında uygulanabilir:

1. Token Enjeksiyonu

Tüm API isteklerine otomatik olarak kimlik doğrulama bilgisi eklenir:

`Authorization: Bearer <token>`

2. Refresh Token Akışı

401 yanıtı alındığında:

1. refresh token ile yeni access token alınır,
2. orijinal istek yeniden gönderilir.

3. Request Standardization

Uygulama genelinde tek tip request formatı sağlar:

```
GET(url, params)
POST(url, body)
PUT(url, body)
DELETE(url)
```

4. Response Mapping (DTO Transformasyonu)

Backend'ten gelen yanıt UI'nin ihtiyaç duyduğu veri şekline dönüştürülür.

5. Error Standardization

Backend farklı hata formatları dökse bile interceptor tek bir ortak hata formatı üretir.

6. Logging ve Analytics

İsteklerin süresi, hatalar ve kullanıcı bağlamı (Sentry, GA, Mixpanel vb.) bu katmanda loglanabilir.

7. Trace-ID Propagation

Frontend tarafında üretilen Trace ID tüm outbound isteklerde backend'e iletilir:

```
X-Trace-Id: <uuid>
```

8. Mock / Staging Yönlendirme

Aynı API çağrısı, environment'a göre farklı backend adreslerine routelanabilir (client-side level routing).

9. Offline-First Mantığı

IndexedDB veya LocalStorage ile cache → network → fallback stratejileri uygulanabilir.

10. Retry ve Backoff Mekanizmaları

Ağ hatalarında client tarafında tekrar deneme politikaları uygulanabilir.

Framework Bazında Örnekler

Angular Interceptor Örneği

```
@Injectable()
export class AuthInterceptor implements HttpInterceptor {
  intercept(req: HttpRequest<any>, next: HttpHandler) {
    const newReq = req.clone({
      setHeaders: {
        Authorization: `Bearer ${token}`,
        'X-Trace-Id': traceId
      }
    });
    return next.handle(newReq);
  }
}
```

React/Vue Axios Interceptor

```
axios.interceptors.request.use((config) => {
  config.headers['X-Trace-Id'] = uuid();
  return config;
});
```

Bu Katmanın Yapamayacakları

Client-side gateway aşağıdaki işlevleri **gerçekleştiremez**:

- Rate limiting veya DDOS koruması
- Gerçek güvenlik politikaları (CORS, mTLS vb.)
- Internal service routing veya service discovery
- Canary deployment veya traffic shaping
- Backend tarafında log, metric veya tracing kontrolü

Dolayısıyla bu katman, API Gateway'in yerini almaz; ancak frontend ile backend arasında mantıksal bir koruma ve standardizasyon katmanı sağlar.

Domain Partitioning

Backend tarafında *domain partitioning*, sipariş, ürün, kullanıcı, ödeme gibi iş alanlarının birbirinden ayrılmış bounded context'ler şeklinde tasarlanması anlamına gelir. Frontend tarafında ise bu kavramın doğrudan karşılığı:

UI, state ve routing yapılarının iş alanlarına (domain boundaries) göre bölünmesidir.

Bu yaklaşım performans, maintainability ve scaling açısından kritiktir.

Frontend'de Domain Partitioning Neden Önemli?

- State tree şişmesini engeller.
- Component'ler arası bağımlılığı azaltır.
- Multi-tab state çakışmalarını önler.
- Routing yapısını sadeleştirir.
- Refactor maliyetini ciddi şekilde düşürür.

Domain partitioning sayesinde UI, bağımsız ve yönetilebilir alt parçalara ayrılmış olur.

Frontend'de Domain Partitioning'in 4 Temel Alanı

State Tree'nin Domain'e Göre Bölünmesi

Angular NgRx yapısında domain-sliced state:

```
store/  
  order/  
    order.actions.ts  
    order.reducer.ts  
    order.selectors.ts  
  user/  
    user.actions.ts  
    user.reducer.ts  
  product/  
    product.reducer.ts
```

React + Redux Toolkit de benzer şekilde:

```
store/  
  orderSlice.js  
  userSlice.js  
  cartSlice.js
```

Her domain kendi reducer, selector ve side-effect yönetimine sahiptir.

Multi-Tab State Yönetimi

Domain partitioning uygulanınca her domain farklı persistence stratejisi alır:

- **auth domain** → global (localStorage + BroadcastChannel)
- **cart domain** → tab-specific (sessionStorage)
- **ui-preferences domain** → persisted (tema, dil)
- **orders domain** → server-source-of-truth

Her domain'in storage politikası farklıdır.

Feature Modules ile Bölme

Angular'da domain partitioning = feature module architecture:

```
src/app/  
  orders/  
  products/  
  auth/  
  dashboard/
```

Her feature module:

- kendi routing'ine sahiptir,
- container/presentational mimarisini içerir,
- kendi NgRx feature store'unu barındırabilir.

Routing'in Domain Sınırlarına Göre Yapılması

Angular routing:

```
/auth/login  
/auth/register  
/products  
/products/:id  
/orders  
/orders/history
```

React Router:

```
<Route path="/orders/*" element={<OrdersPage />} />  
<Route path="/products/*" element={<ProductsPage />} />
```

Domain boundaries, lazy loading, security, permission ve code-splitting davranışlarını belirler.

Encapsulation ile İlgili Temel Yazılım Kavramları

Kavram	Kısa Tanım	Encapsulation ile İlişkisi
Abstraction	Gereksiz detayları sadeleştirme, konsepti basitleştirme.	Encapsulation detayları gizler; abstraction detayları azaltır. Yakın ama farklı kavramlardır.
Information Hiding	İç mantığı, veriyi ve implemantasyonu saklama (private, protected).	Information Hiding, encapsulation'ın alt kümesidir; gizleme yönünü temsil eder.
Modularity	Sistemin bağımsız ve değiştirilebilir parçalara bölünmesi.	Modüller arası sınırlar encapsulation sayesinde belirginleşir; birlikte çalışırlar.
Composition	Küçük parçaların bir araya gelerek büyük davranış oluşturmaları.	Composition, encapsulation'ı destekler; inheritance'a göre daha sağlıklı yapılar kurar.
Inheritance	Bir sınıfın başka bir sınıftan miras alması.	Inheritance çoğu zaman encapsulation'ı kırar çünkü alt sınıf üst sınıfın içeriğini bilmek zorunda kalabilir.
Coupling	Modüllerin birbirine bağımlılık derecesi.	Güçlü encapsulation, düşük coupling yaratır. Kötü encapsulation → yüksek coupling.
Cohesion	Bir modülün kendi işine ne kadar odaklandığı, iç tutarlılık.	Yüksek cohesion + güçlü encapsulation temiz tasarımın temelidir.
Interfaces	Dış dünyaya açılan kontrat (public API).	Encapsulation nesnenin içeriğini saklar; interface dış sınırı çizer. Birlikte çalışırlar.
Dependency Injection	Bağımlılıkların nesneye dışarıdan verilmesi.	DI encapsulation'ı güçlendirir; sınıf dış bağımlılıkları kendi üretmez, dışa bağımlı olmaz.
State Management	Uygulamanın durumunun merkezi bir store'da yönetilmesi (NgRx, Redux).	Encapsulation mikro-level state düzenlerken, state management macro-level state'i yönetir; tamamlayıcıdır.
Immutability	State'in değiştirilemez olması.	Encapsulation state'i korur; immutability state'in nasıl değiştirileceğini kısıtlar.
Polymorphism	Aynı interface üzerinden farklı davranışların gerçekleşmesi.	Encapsulation davranışı içte tutar, polymorphism davranış varyasyonlarını dış dünyaya sunar.
API Design	Bir modülün hangi fonksiyonları dışa açacağını belirlemek.	Encapsulation neyin public neyin private olacağını belirleyerek API yüzeyini tanımlar.
Access Modifiers	public, private, protected, read-only gibi görünürlük belirleyiciler.	Encapsulation'ın uygulanmasını sağlayan temel araçlardır.
Encapsulation	Veri ve davranışın bir sınıfın içinde tutulması; iç detayların dışarıya kapatılması.	External API yüzeyini kontrol ederek modüller arası sınır çizer; Information Hiding ve Access Modifiers ile uygulanır.

İlişkili Kavramlar

1. Abstraction

Gereksiz detayları azaltarak konsepti sadeleştirme. Angular'da servisler, bileşenlerden altyapı detaylarını gizleyerek soyutlama sağlar.

```
export class UserService {  
  getUser() {  
    return this.http.get('/api/user'); // detay gizlendi  
  }  
}  
  
this.userService.getUser().subscribe();
```

2. Information Hiding

Gizlenmesi gereken iç mantığın `private` ya da `protected` ile saklanması.

```
export class AuthService {  
  private token: string | null = null;  
  
  setToken(t: string) {  
    this.token = t;  
  }  
}
```

3. Modularity

Uygulamanın bağımsız feature modüllerine bölünmesi.

```
auth/  
  login/  
  register/  
  auth.module.ts
```

4. Composition

Büyük ekranların küçük, yeniden kullanılabilir bileşimlerle oluşturulması.

```
<app-header></app-header>  
<app-user-card></app-user-card>  
<app-appointment-list></app-appointment-list>
```

5. Inheritance

Inheritance, bir sınıfın başka bir sınıfın özelliklerini ve davranışlarını **miras alması** anlamına gelir.

Kısaca:

“Alt sınıf, üst sınıfın bir çeşididir.”

Bu model, ortak davranışların tekrar edilmeden paylaşılmasını sağlar.

Günlük Hayattan Basit Analoji

Serçe bir kuşun alt türüdür. Bu nedenle “kuş” a ait uçma, ötme gibi davranışları miras alır. Üst sınıf ortak davranışları barındırır; alt sınıf bu davranışları otomatik olarak edinir.

```
export abstract class BaseFormComponent {
  isSubmitting = false;

  submit() {
    this.isSubmitting = true;
    console.log("form submitted");
  }
}

export class LoginComponent extends BaseFormComponent {
  login() {
    this.submit(); // üst sınıftaki metodu kullanır
  }
}
```

Bu durumda LoginComponent, BaseFormComponent’in özelliklerine ve metodlarına sahip olur.

Neden Bazen Sorunlu?

Inheritance çoğu zaman encapsulation’ı zayıflatır. Bunun temel nedenleri:

- Alt sınıf, üst sınıfın iç detaylarını bilmek zorunda kalır.
- Üst sınıfa yapılan küçük bir değişiklik alt sınıfları bozabilir.
- Üst sınıf çok fazla sorumluluk taşıdığında “kümülatif karmaşıklık” oluşur.

6. Coupling

Bağımlılık derecesi. Kötü örnek: doğrudan sınıf üretmek. İyi örnek: Dependency Injection.

```
// kötü: yüksek coupling
const service = new AppointmentService();

// iyi: düşük coupling
constructor(private service: AppointmentService) {}
```

7. Cohesion

Bir bileşenin tek iş yapması yüksek cohesion sağlar.

```
LoginComponent
UserListComponent
LogoutButtonComponent
```

8. Interfaces

API'nin dış sözleşmesi.

```
export interface IUser {
  id: number;
  name: string;
  email: string;
}
```

9. Dependency Injection

Bağımlılıkların bileşene dışarıdan enjekte edilmesi.

```
constructor(private http: HttpClient) {}
```

10. State Management (NgRx)

Component mikro-state'i kapsüller, store ise makro-state yönetir.

```
this.store.dispatch(loadUser());

export const reducer = createReducer(
  initialState,
  on(loadUserSuccess, (s, { user }) => ({ ...s, user })))
);
```

11. Immutability

State doğrudan değiştirilmez, yeni bir kopya üretilir.

```
return {
  ...state,
  items: [...state.items, newItem]
}
```

12. Polymorphism

Aynı interface ile farklı davranış sergileyen sınıflar.

```
export interface FieldRenderer {
  render(): void;
}

export class TextRenderer implements FieldRenderer { render() {} }
export class CheckboxRenderer implements FieldRenderer { render() {} }
```

13. API Design

Public API yüzeyini minimal tutarak kapsülleme sınırı çizilir. Gerçek projelerde bazı componentlerin domain gereği giderek büyümesi ve parçalanamaması mümkündür. Böyle bir durumda componentin `@Input()` sayısının 10 ve üzerine çıkması, API yüzeyini karmaşık hale getirir ve bakım maliyetini artırır. Bu senaryoda amaç, componenti yeniden yapılandırmadan API tasarımını sadeleştirmektir. Aşağıdaki dört yöntem, Angular dünyasında yaygın olarak kabul görmüş çözümlerdir.

1. Config Object Pattern (En İyi Çözüm)

Birden fazla Input'u tek bir `config` objesinde toplamak, component API'sini sadeleştirir ve "public surface area"yı küçültür.

Kötü API (10+ Input):

```
<app-report
  [startDate]="start"
  [endDate]="end"
  [type]="type"
  [format]="format"
  [showHeader]="true"
  [showFooter]="false"
  [filters]="filters"
  [theme]="theme"
  [size]="size"
  [layout]="layout"
></app-report>
```

İyi API (Tek Bir Config):

```
<app-report [config]="reportConfig"></app-report>

@Input() config!: ReportConfig;

export interface ReportConfig {
  startDate: Date;
  endDate: Date;
  type: 'pdf' | 'excel' | 'html';
  format: string;
  showHeader: boolean;
  showFooter: boolean;
  filters: Filter[];
  theme: Theme;
  size: Size;
  layout: Layout;
}
```

Bu yaklaşım, API'yi sadeleştirir, bilişsel yükü azaltır ve componenti yeniden kullanılabilir hale getirir.

2. Input Grouping (Anlamsal Gruplama)

Tek bir `config` objesi çok büyüdüğünde (örneğin 12-25 alan içerdiğinde), Input'ların anlamsal olarak bölünmesi gereklidir. Bu işlem, component API'sinin daha okunabilir, daha tahmin edilebilir ve daha domain odaklı olmasını sağlar. Amaç, farklı kavramsal sorumlulukları birbirinden ayırarak “mini-config” yapıları oluşturmaktır.

Bu ayrım genellikle üç temel kategori üzerinden yapılır:

- **Data Config** — Component'e verilen iş verisi (model, filtreler, domain alanları)
- **Display Config** — Component'in nasıl görüneceği (tema, layout, görünürlük ayarları)
- **Behavior Config** — Component'in nasıl davranacağı (etkileşimler, event kuralları)

Bu kategoriler, aşağıdaki örnekte olduğu gibi ayrı Input'lara bölünür:

```
<app-report
  [data]="dataConfig"
  [display]="displayConfig"
  [behavior]="behaviorConfig"
></app-report>
```

1) Data Config (İş Verisi Yapılandırması) Component’in çalışması için gerekli domain verileri burada tanımlanır.

```
export interface ReportDataConfig {
  startDate: Date;
  endDate: Date;
  filters: Filter[];
  type: 'pdf' | 'excel' | 'html';
}
```

2) Display Config (Görsel Ayarlar) Görünüm, tema ve kullanıcıya sunulan UI özellikleri bu gruptadır.

```
export interface ReportDisplayConfig {
  theme: Theme;
  size: 'small' | 'medium' | 'large';
  layout: 'compact' | 'spacious';
  showHeader: boolean;
  showFooter: boolean;
}
```

3) Behavior Config (Davranışsal Ayarlar) Component’in kullanıcı etkileşimlerine nasıl tepki vereceği burada tanımlanır.

```
export interface ReportBehaviorConfig {
  autoRefresh: boolean;
  refreshInterval?: number;
  enableClickTracking: boolean;
  disableAnimations: boolean;
}
```

Toparlayıcı Örnek Tüm konfigürasyonlar, component içinde ayrı ayrı Input olarak alınır:

```
@Input() data!: ReportDataConfig;
@Input() display!: ReportDisplayConfig;
@Input() behavior!: ReportBehaviorConfig;
```

Bu Yapının Avantajları

- Component API’si okunabilir hale gelir.
- Sorumluluklar net biçimde ayrılır.
- Domain’e uygun “mini-config” modelleri gelişir.
- Test edilmesi ve genişletilmesi kolaylaşır.
- 15 Input yerine 3 adet anlamlı Input kullanılmış olur.

3. Zorunlu Input'lar İçin Factory Service Kullanımı

Bazı componentlerde çok sayıda `@Input()` alanı bulunabilir ve bu alanların bir kısmı domain gereği **zorunlu** olabilir. Bu durumda tüm zorunlu Input'ları doğrudan component API'sine koymak, API yüzeyini büyütür, component'i kırılgan hale getirir ve yanlış kullanım riskini artırır. Bu problemi çözmek için "Factory Service" yaklaşımı kullanılır.

Amaç, component'e geçilmeden önce:

- zorunlu alanların bir araya getirilmesi,
- eksik alanların kontrol edilmesi,
- default değerlerin atanması,
- verinin normalize edilmesi

gibi işlemleri *tek bir merkezde toplamak* ve component'e sadece **geçerli bir konfigürasyon objesi** göndermektir.

Factory Service Mantığı Component'i kullanan taraf, zorunlu parametreleri bir servis üzerinden oluşturur:

```
constructor(private reportFactory: ReportFactory) {}

reportConfig = this.reportFactory.create({
  startDate,
  endDate,
  type: 'pdf'
});
```

Burada `create()` metodu:

- zorunlu alanların varlığını doğrular,
- default değerleri ekler,
- domain kurallarını uygular,
- ortaya tamamen geçerli bir `ReportConfig` objesi çıkarır.

Factory Service Örneği

```
create(data: Partial<ReportConfig>): ReportConfig {
  if (!data.startDate) throw new Error('startDate gerekli');
  if (!data.endDate) throw new Error('endDate gerekli');

  return {
    type: data.type ?? 'pdf',
    showHeader: true,
```

```

    showFooter: true,
    ...data
  };
}

```

Bu sayede component, yalnızca **hazır ve doğrulanmış** bir config alır.

Component API’sinin Sadeleştirilmiş Hali

```
<app-report [config]="reportConfig" [filters]="filters"></app-report>
```

Component yalnızca minimal Input’lar kabul eder:

```

@Input() config!: ReportConfig;
@Input() filters?: Filter[];

```

Avantajları

- Component API’si küçülür ve daha okunabilir hale gelir.
- Tüm zorunlu alanlar tek merkezden yönetildiği için hata riski azalır.
- Domain kuralları component’ten ayrılarak “single responsibility” korunur.
- Config, uygulamanın birçok yerinde tekrar kullanılabilir.
- API Design prensiplerine uygun minimal bir public surface oluşturulur.

Özetle, Factory Service yaklaşımı, büyük ve karmaşık componentlerde zorunlu Input yönetimini sadeleştirerek component’i daha güvenli, modüler ve bakımı kolay bir hale getirir.

4. Davranış Odaklı Input’ların Directive’e Taşınması

Bazı @Input() alanları, componentin görünümüyle değil **davranışıyla** ilgilidir. Örneğin:

- showTooltip
- clickOutsideToClose
- highlightInvalid
- disableAnimations

Bu tür davranışlar component API’sini şişirir ve componentin asıl sorumluluğu olan “UI render etme” görevinden uzaklaşmasına neden olur. Bu nedenle davranış odaklı Input’lar component’ten ayrılarak **directive** olarak uygulanmalıdır.

Yanlış Kullanım (Davranış Component'e Ekleme)

```
<app-report
  [clickOutsideToClose]="true"
  [showTooltip]="true"
></app-report>
```

Bu yaklaşım:

- component API'sini gereksiz şekilde büyütür,
- component'in davranış yükünü artırır,
- tekrar kullanılabilirliği azaltır.

Doğru Kullanım (Davranış Directive'e Taşımak)

```
<app-report appClickOutside appTooltip="Report info"></app-report>
```

Directive Örneği

```
@Directive({
  selector: '[appClickOutside]',
})
export class ClickOutsideDirective {
  @HostListener('document:click', ['$event'])
  onClick(event: Event) {
    // click outside behavior burada yönetilir
  }
}
```

Bu sayede:

- component'in Input sayısı azalır,
- davranış yönetimi ayrı bir sorumluluk olarak yalıtılır,
- aynı davranış birden fazla component'te tekrar kullanılabilir,
- component daha sade ve daha kolay test edilebilir hale gelir.

Temiz API tasarımı için ideal yaklaşım, mümkün olduğunca **component içinde zaten mevcut veya türetilabilir bilgiyi kullanmak**, sadece zorunlu olan değerleri dışarıdan almak ve API'yi minimal tutmaktır. Aşağıdaki teknikler, dışarıdan gereksiz Input vermek yerine component içinde veri kullanarak daha sade bir API oluşturmayı sağlar.

İç State Kullanımı (Component İçinden Bilgi Elde Etme)

Bir component, bazı bilgileri zaten kendi içinden veya servislerden elde edebiliyorsa, bu değerler Input olarak dışarıdan alınmamalıdır.

Örnek

```
private userId = this.auth.getCurrentUserId();
private theme = this.themeService.currentTheme();
private mode = 'compact';
```

Bu durumda component'in hiçbir Input'a ihtiyacı yoktur:

```
<app-report></app-report>
```

Türetilmiş Verileri Input Olarak Almamak

Eğer bir değer component içinde türetilbiliyorsa (örneğin “bugün” için tarih aralığı), bunun Input olarak verilmesi gereksizdir.

Örnek

```
get startOfDay() {
  return new Date().setHours(0,0,0,0);
}
```

Service'leri Input Olarak Vermek Yerine Dependency Injection Kullanmak

Service nesnelerinin Input olarak geçirilmesi kötü bir uygulamadır. Angular'ın DI mekanizması zaten component'e gerekli servisleri sağlar.

Yanlış Kullanım

```
<app-profile [userService]="userService"></app-profile>
```

Doğru Kullanım

```
constructor(private userService: UserService) {}
```

Context-Based API (Parent/Ancestor State Kullanımı)

Component hiyerarşisinde bazı bilgiler zaten üst component'te bulunabilir. Bu durumda aynı veri çocuk component'e Input olarak verilmek zorunda değildir.

Yanlış Kullanım

```
<app-header [id]="appointmentId"></app-header>
<app-details [id]="appointmentId"></app-details>
```

Doğru Kullanım (Parent State Erişimi)

```
constructor(@Host() private parent: AppointmentPage) {
  this.appointmentId = parent.appointmentId;
}
```


Angular’da Context Kavramı (React Context API ile Karıştırılmamalıdır)

Bu bölümde kullanılan *context* terimi, React’teki **Context** API ile aynı yapıyı ifade etmez. React’te Context, global veya paylaşılan durum yönetimi için kullanılan özel bir mekanizmadır (Provider → Consumer modeli). Angular’da ise “context” daha farklı ve daha basit bir anlam taşır.

Angular’daki Context Nedir? Angular ekosisteminde context, bir component’in **component ağacındaki konumu** ve bu konumdan dolayı erişebildiği **doğal bağlam** anlamına gelir. Bu bağlam şunları içerir:

- parent component’in mevcut state’i,
- ancestor (üst düzey) component’lerin verileri,
- component hiyerarşisinin doğal veri ortamı,
- Angular’ın Dependency Injection mekanizmasıyla erişilebilen üst component instance’ları.

Bu nedenle Angular’da context, bir API veya özel bir mekanizma değil; component ağacının sağladığı doğal veri erişim yoludur.

- React: Global/paylaşılan state taşımak için özel bir Context API kullanılır.
- Angular: Parent-child ilişkisiyle oluşan doğal bağlam ve DI sistemi kullanılır.

Angular’da “context” kelimesi yalnızca “component’in bulunduğu çevre ve erişebildiği üst component state’i” anlamına gelir.

Örnek: Parent State’ine Context Yoluyla Erişim

```
export class AppointmentPage {  
  appointmentId = this.route.snapshot.params['id'];  
}
```

Child component bu veriyi Input olarak istemek yerine, parent context’inden çekebilir:

```
constructor(@Host() private parent: AppointmentPage) {  
  this.appointmentId = parent.appointmentId;  
}
```

Şişmiş Constructor (Constructor Injection Overload) ve Çözüm Teknikleri

Angular component ve servislerinde kullanılan Dependency Injection mekanizması, aşırı sayıda bağımlılık eklendiğinde “şişmiş constructor” problemine yol açabilir. Bu durum hem API tasarımının bozulmasına hem de ilgili sınıfın tek sorumluluk ilkesinden (*Single Responsibility Principle*) uzaklaşmasına neden olur.

Şişmiş Constructor Nedir? Aşağıdaki örnek, component’in gereğinden fazla bağımlılığa sahip olduğunu gösterir:

```
constructor(  
  private auth: AuthService,  
  private api: ApiService,  
  private logger: LoggerService,  
  private theme: ThemeService,  
  private state: AppStateService,  
  private cache: CacheService,  
  private permissions: PermissionService,  
  private events: EventBusService,  
) {}
```

Bu yapı:

- okunabilirliği düşürür,
- test yazmayı zorlaştırır (çok sayıda mock gerektirir),
- sınıfın çok fazla sorumluluk taşıdığını gösterir,
- yüksek coupling oluşturur.

Dolayısıyla şişmiş constructor, özellikle UI component’lerinde **tolere edilemez**.

Şişmeyi Önleyen Beş Temel Teknik Aşağıdaki teknikler, constructor şişmesini tamamen engeller ve API yüzeyini sade tutar.

1) Facade Pattern (En Etkili Çözüm)

Çok sayıda servis, tek bir “Facade” sınıfında birleştirilerek component yalnızca bu facade’ı inject eder.

Kötü Örnek

```
constructor(  
  private user: UserService,  
  private role: RoleService,  
  private perm: PermissionService  
) {}
```

İyi Örnek

```
constructor(private securityFacade: SecurityFacade) {}

export class SecurityFacade {
  constructor(
    private user: UserService,
    private role: RoleService,
    private perm: PermissionService
  ) {}
}
```

2) Domain Service Consolidation

Aynı domain'e ait birden fazla servis tek bir domain serviste toplanır.

```
AppointmentStateService
AppointmentApi
AppointmentCache
AppointmentValidator
```

Bu servisler tek bir AppointmentService altında birleştirilebilir.

3) Üst Seviye (Module-Level) DI Kullanımı

Bazı servisler doğrudan component tarafından yönetilmesi gereken bağımlılıklar değildir. Bu servisler, uygulama genelinde (root veya module seviyesinde) tek bir instance olarak sağlanabilir. Bu yaklaşımla component'in constructor'ına gereksiz bağımlılık eklenmez ve constructor'ın "şişmesi" önlenir.

Örnek: Gereksiz Component-Level Injection

```
constructor(private logger: LoggerService) {}
```

Burada LoggerService, component'in asıl sorumluluğunun bir parçası olmadığı halde component'e enjekte edilmektedir. Bu durum component'in bağımlılık yükünü artırır.

Doğru Yaklaşım: Root-Level Provider

```
@Injectable({
  providedIn: 'root'
})
export class LoggerService {}
```

Bu tanımlama ile Angular, LoggerService için uygulama genelinde tek bir instance oluşturur. Component bu servisi doğrudan kullanmak zorunda değilse constructor'a eklemeyiz; böylece component API'si sade kalır.

Sonuç Module-level (veya root-level) DI, komponentlerin yalnızca gerçekten ihtiyaç duydukları bağımlılıkları constructor’a eklemesini sağlar. Uygulama genelinde kullanılan servisler üst seviyede sağlandığında, component constructor’ı daha temiz, daha kısa ve daha yönetilebilir olur.

4) Store/Facade Kullanımı (NgRx, Signals)

Çok sayıda bağımlılık yerine tek bir store veya facade üzerinden gerekli veri ve event erişimi sağlanır.

```
constructor(private appointmentFacade: AppointmentFacade) {}
```

5) Manager / Coordinator Sınıfları

Component’in orkestrasyon yükü fazla ise, bileşen sadece tek bir “Manager” sınıfını inject eder. Manager sınıfı ağır olabilir; bu normaldir çünkü orkestrasyon sorumluluğu ondadır.

```
constructor(private dashboardManager: DashboardManager) {}
```

Manager Sınıfı Örneği

```
export class DashboardManager {
  constructor(
    private metrics: MetricsService,
    private charts: ChartService,
    private filters: FilterService,
    private logger: LoggerService,
    private api: DashboardApi,
  ) {}
}
```

5. Default Değerler ve Config Object Kullanımı

Bazı componentler default ayarlar üzerinden çalışabilir. Bu durumda Input vermek zorunda kalmadan iç state kullanılabilir.

Örnek

```
defaultConfig = {
  size: 'md',
  backdrop: true,
  closeOnEsc: true
};

<app-dialog></app-dialog>
```

6. Derived State Pattern

Bazı değerler, mevcut state üzerinden türetilir. Bu tür bilgilerin Input olarak verilmesi yerine component içinde hesaplanması daha doğrudur.

Input Vermek Gereksizdir

```
<app-report [expired]="isExpired()"></app-report>
```

Doğru Yaklaşım

```
isExpired() {  
  return this.config.endDate < new Date();  
}
```

Sonuç

Dışarıdan gereksiz sayıda Input almak yerine component içinde mevcut, türetilir veya servislerden edinilebilir veriyi kullanmak; API yüzeyinin küçülmesini, componentin daha sade, daha öngörülebilir ve daha yeniden kullanılabilir olmasını sağlar. Bu yaklaşım, temiz API tasarımının temel taşlarından biridir.

Access Modifiers

Access Modifiers (erişim belirleyiciler), TypeScript ve nesne yönelimli dillerde bir sınıfın hangi üyelerinin dışarıdan erişilebilir olacağını belirleyen mekanizmalardır. Bu yapı, **encapsulation** ilkesinin temel uygulama aracıdır; yani sınıfın iç detaylarını saklamaya ve dış dünyaya yalnızca gerekli olan kısmı açmaya yarar.

TypeScript'te üç temel erişim belirleyici bulunur:

- **public** — Her yerden erişilebilir.
- **private** — Yalnızca sınıfın kendi içinde erişilebilir.
- **protected** — Sınıfın kendisi ve ondan türeyen alt sınıflar tarafından erişilebilir.

Örnek Kod

```
class UserService {  
  public login() {} // Dışarıya açık API  
  
  private token = ""; // Yalnızca sınıf içinde görülebilir  
  
  protected refresh() {} // Bu sınıftan türeyen alt sınıflar görebilir  
}
```

Açıklamalar

- `public login()` — Component'ler, servisler veya uygulamanın herhangi bir başka bölümü tarafından çağrılabilir. Bu, sınıfın dış dünyaya açtığı “public API” dir.
- `private token` — Tamamen gizlenmiş bir alandır. Ne component'ler ne de alt sınıflar bu değişkene erişebilir. Bu mekanizma, **information hiding** ilkesinin doğrudan karşılığıdır.
- `protected refresh()` — Yalnızca `UserService` veya ondan `extends` ile türeyen alt sınıflar tarafından kullanılabilir. Bu genellikle “ortak fakat dışarı açılmayan” davranışların paylaşılmasında kullanılır.

Dashboard Component (Parçalanmamış Kötü Tasarım)

Aşağıdaki örnek, pek çok sorumluluğu aynı anda üstlenen ve constructor ile component sınıfının gereksiz derecede şişmesine yol açan kötü bir Angular component tasarımını göstermektedir. Bu yapı, daha sonra Smart/Dumb Component, Manager/Coordinator Pattern ve Input ayrıştırma teknikleriyle parçalanacaktır.

dashboard.component.ts

```
import { Component, OnInit } from '@angular/core';
import { FormBuilder, Validators } from '@angular/forms';
import { DashboardApi } from '../services/dashboard.api';
import { MetricsService } from '../services/metrics.service';
import { LoggerService } from '../services/logger.service';
import { ChartService } from '../services/chart.service';
import { FilterService } from '../services/filter.service';
import { BehaviorSubject, combineLatest } from 'rxjs';
import { map, tap, switchMap } from 'rxjs/operators';

@Component({
  selector: 'app-dashboard',
  templateUrl: './dashboard.component.html',
  styleUrls: ['./dashboard.component.css']
})
export class DashboardComponent implements OnInit {
  loading = false;
  error: string | null = null;

  // UI state
  selectedRange$ = new BehaviorSubject<'today' | 'week' | 'month'>('today');

  // Form
  filterForm = this.fb.group({
    search: [''],
    minValue: [0, Validators.min(0)],
    maxValue: [100, Validators.max(100)],
  });

  dashboardData$ = combineLatest([
    this.selectedRange$,
    this.filterForm.valueChanges
  ]).pipe(
    tap(() => {
      this.loading = true;
      this.error = null;
      this.logger.info('Dashboard loading triggered.');
```

```

    })),
    switchMap(([range, filters]) =>
      this.api.getDashboard(range, filters).pipe(
        map(res => ({
          ...res,
          chart: this.chartService.buildChart(res.items)
        })),
        tap({
          next: () => {
            this.metrics.trackLoad(range);
            this.loading = false;
          },
          error: (err) => {
            this.logger.error('Dashboard error', err);
            this.loading = false;
            this.error = 'Error loading dashboard';
          }
        })
      )
    )
  );

  constructor(
    private fb: FormBuilder,
    private api: DashboardApi,
    private metrics: MetricsService,
    private logger: LoggerService,
    private chartService: ChartService,
    private filterService: FilterService,
  ) {}

  ngOnInit() {
    this.filterService.initialize();
    this.filterForm.updateValueAndValidity();
    this.logger.info('Dashboard component initialized.');
```

```

  }

  selectRange(range: 'today' | 'week' | 'month') {
    this.selectedRange$.next(range);
  }

```

```

  resetFilters() {
    this.filterForm.patchValue({
      search: '',
      minValue: 0,
      maxValue: 100 }); } }

```


dashboard.component.html

```
<div class="dashboard-container">

  <h1>Dashboard</h1>

  <div class="filter-section">
    <form [formGroup]="filterForm">
      <input type="text" placeholder="Search..." formControlName="search" />
      <input type="number" formControlName="minValue" />
      <input type="number" formControlName="maxValue" />
      <button type="button" (click)="resetFilters()">Reset</button>
    </form>

    <div class="range-buttons">
      <button (click)="selectRange('today')">Today</button>
      <button (click)="selectRange('week')">This Week</button>
      <button (click)="selectRange('month')">This Month</button>
    </div>
  </div>

  <div *ngIf="loading" class="loading">Loading...</div>
  <div *ngIf="error" class="error">{{ error }}</div>

  <ng-container *ngIf="dashboardData$ | async as data">
    <app-chart [data]="data.chart"></app-chart>

    <div class="items">
      <div class="item" *ngFor="let item of data.items">
        <h3>{{ item.title }}</h3>
        <p>{{ item.description }}</p>
        <span>{{ item.value }}</span>
      </div>
    </div>
  </ng-container>

</div>
```

Dashboard Component (Parçalanmış Tasarım)

1. Adım: Manager / Coordinator Pattern Uygulaması

Dashboard component, başlangıç hâlinde API çağrıları, chart oluşturma, metrics takibi, error handling, reactive pipeline yönetimi, filtre başlatma ve UI etkileşimlerinin tamamını tek sınıf içinde barındırmaktadır. Bu durum hem constructor'ın şişmesine hem de component'in aşırı sorumluluk üstlenmesine yol açmaktadır. Bu problemi çözmek için tüm orchestration işlemleri ayrı bir "Manager" sınıfına taşınır.

Aşağıda **DashboardManager** sınıfı ve refactor edilmiş **DashboardComponent** yer almaktadır.

DashboardManager (Tüm Orchestration'ın Taşındığı Sınıf)

```
import { Injectable } from '@angular/core';
import { DashboardApi } from '../services/dashboard.api';
import { MetricsService } from '../services/metrics.service';
import { LoggerService } from '../services/logger.service';
import { ChartService } from '../services/chart.service';
import { FilterService } from '../services/filter.service';
import { BehaviorSubject, combineLatest, Observable } from 'rxjs';
import { map, tap, switchMap } from 'rxjs/operators';
import { FormGroup } from '@angular/forms';

@Injectable()
export class DashboardManager {

  private selectedRange$ = new BehaviorSubject<'today' | 'week' | 'month'>('today');

  constructor(
    private api: DashboardApi,
    private metrics: MetricsService,
    private logger: LoggerService,
    private chart: ChartService,
    private filters: FilterService
  ) {}

  initialize(filterForm: FormGroup) {
    this.filters.initialize();
    filterForm.updateValueAndValidity();
    this.logger.info('DashboardManager initialized.');
```

```
}
```

```
setRange(range: 'today' | 'week' | 'month') {
  this.selectedRange$.next(range);
}
```

```
loadDashboard(filterForm: FormGroup): Observable<any> {
  return combineLatest([
    this.selectedRange$,
    filterForm.valueChanges
  ]).pipe(
    tap(() => {
      this.logger.info('Dashboard loading triggered.');
```

```
}),
```

```
switchMap(([range, filters]) =>
```

```
  this.api.getDashboard(range, filters).pipe(
    map(res => ({
```

```

        ...res,
        chart: this.chart.buildChart(res.items)
    )),
    tap({
        next: () => this.metrics.trackLoad(range),
        error: (err) => this.logger.error('Dashboard error', err)
    })
    )
    );
}
}

```

Açıklama: Bu sınıf, daha önce component içinde bulunan tüm ağır işleri üstlenmiştir: API çağrıları, chart oluşturma, metrics takibi, filter init, error logging, reactive stream kontrolü ve range yönetimi artık tek bir merkezde toplanmaktadır.

Refactor Edilmiş DashboardComponent (Hafifletilmiş Smart Component)

```
import { Component, OnInit } from '@angular/core';
import { FormBuilder, Validators } from '@angular/forms';
import { DashboardManager } from '../dashboard.manager';

@Component({
  selector: 'app-dashboard',
  templateUrl: './dashboard.component.html',
  styleUrls: ['./dashboard.component.css'],
  providers: [DashboardManager]
})
export class DashboardComponent implements OnInit {

  loading = false;
  error: string | null = null;

  filterForm = this.fb.group({
    search: [''],
    minValue: [0],
    maxValue: [100],
  });

  dashboardData$ = this.manager.loadDashboard(this.filterForm);

  constructor(
    private fb: FormBuilder,
    private manager: DashboardManager
  ) {}

  ngOnInit() {
    this.manager.initialize(this.filterForm);
  }

  selectRange(range: 'today' | 'week' | 'month') {
    this.manager.setRange(range);
  }

  resetFilters() {
    this.filterForm.patchValue({
      search: '',
      minValue: 0,
      maxValue: 100
    });
  }
}
```

Açıklama: Refactor sonrasında component yalnızca:

- form state yönetimini,
- UI etkileşimlerini,
- manager metotlarını çağırmaı,
- template'e veri sağlamayı

üstlenmektedir. Component artık hiçbir iş kuralı, API akışı, chart üretimi, metrics takibi veya logging sorumluluğu taşımamaktadır. Constructor sadece iki bağımlılığa düşmüştür.

Sonuç: Manager/Coordinator Pattern sayesinde component aşırı sorumluluk-tan arındırılmış, orchestration tek merkezde toplanmış ve component API'si kayda değer şekilde sadeleşmiştir. Bu yapı sistemin test edilebilirliğini, bakım kolaylığını ve okunabilirliğini ciddi ölçüde artırır.

2. Adım: Smart–Dumb Component Ayrımı

Manager/Coordinator Pattern uygulandıktan sonra component’in iş yükü önemli ölçüde azalmıştır. Ancak HTML tarafı hâlâ Smart Component içinde durmakta ve UI render lojigi ile container mantigi karismaktadır. Bu aşamada component ikiye ayrilir:

- **Smart Component (Container)** — veri akisini yönetir, Manager ile konusur.
- **Dumb Component (Presentational)** — yalnızca UI render eder, hiç iş mantigi içermez.

Bu ayrım, UI ile business logic arasındaki bağı tamamen koparmayı ve component mimarisi içinde “Single Responsibility Principle”in eksiksiz uygulanmasını sağlar.

DashboardContainer (Smart Component)

Smart component yalnızca:

- Manager’dan veri alır,
- form state’ini yönetir,
- UI’ya taşınacak veriyi Dumb Component’e iletir,
- herhangi bir UI render mantigi içermez.

```

@Component({
  selector: 'app-dashboard',
  template: `
    <app-dashboard-view
      [data]="dashboardData$ | async"
      [loading]="loading"
      [error]="error"
      (rangeSelected)="selectRange($event)"
      (resetRequested)="resetFilters()"
      [form]="filterForm">
    </app-dashboard-view>
  `,
  styleUrls: ['./dashboard.component.css'],
  providers: [DashboardManager]
})
export class DashboardComponent implements OnInit {

  loading = false;
  error: string | null = null;

  filterForm = this.fb.group({
    search: [''],
    minValue: [0],
    maxValue: [100],
  });

  dashboardData$ = this.manager.loadDashboard(this.filterForm);

  constructor(
    private fb: FormBuilder,
    private manager: DashboardManager
  ) {}

  ngOnInit() {
    this.manager.initialize(this.filterForm);
  }

  selectRange(range: 'today' | 'week' | 'month') {
    this.manager.setRange(range);
  }

  resetFilters() {
    this.filterForm.patchValue({
      search: '',
      minValue: 0,
      maxValue: 100 }); } }

```


DashboardView (Dumb Component)

Dumb Component:

- yalnızca Input alır,
- yalnızca Output event üretir,
- veri toplamaz, API çağırılmaz, Manager ile konuşmaz,
- sadece UI render eder.

dashboard-view.component.ts

```
@Component({
  selector: 'app-dashboard-view',
  templateUrl: './dashboard-view.component.html',
  styleUrls: ['./dashboard-view.component.css']
})
export class DashboardViewComponent {

  @Input() data: any;
  @Input() form!: FormGroup;
  @Input() loading!: boolean;
  @Input() error!: string | null;

  @Output() rangeSelected = new EventEmitter<'today' | 'week' | 'month'>();
  @Output() resetRequested = new EventEmitter<void>();
}
```

dashboard-view.component.html

```
<div class="dashboard-container">

  <h1>Dashboard</h1>

  <div class="filter-section">
    <form [formGroup]="form">
      <input type="text" placeholder="Search..." formControlName="search" />
      <input type="number" formControlName="minValue" />
      <input type="number" formControlName="maxValue" />
      <button type="button" (click)="resetRequested.emit()">Reset</button>
    </form>

    <div class="range-buttons">
      <button (click)="rangeSelected.emit('today')">Today</button>
      <button (click)="rangeSelected.emit('week')">This Week</button>
      <button (click)="rangeSelected.emit('month')">This Month</button>
    </div>
  </div>

  <div *ngIf="loading" class="loading">Loading...</div>
  <div *ngIf="error" class="error">{{ error }}</div>

  <ng-container *ngIf="data">
    <app-chart [data]="data.chart"></app-chart>

    <div class="items">
      <div class="item" *ngFor="let item of data.items">
        <h3>{{ item.title }}</h3>
        <p>{{ item.description }}</p>
        <span>{{ item.value }}</span>
      </div>
    </div>
  </ng-container>

</div>
```

Sonuç: Bu adım sonucunda:

- Component render mantığı tamamen ayrıştırılmıştır,
- Smart Component yalnızca veri akışını yönetmektedir,
- Dumb Component yalnızca UI ile ilgilenmektedir,
- Manager + Smart + Dumb tam bir “çok katmanlı UI mimarisi” oluşturmuştur.

Cross-Cutting Concerns

Modern yazılım mimarisinde **cross-cutting concern**, tek bir modüle, sınıfa veya fonksiyona ait olmayan; sistemin pek çok farklı noktasında ortak biçimde ihtiyaç duyulan ancak doğrudan iş mantığının bir parçası olmayan sorumlulukları ifade eder.

Bu tür sorumluluklar, sistemde yatay olarak yayıldıkları ve birden fazla katmanı “yanlamasına kestikleri” için cross-cutting (yatay kesen) olarak adlandırılır.

Başlıca Cross-Cutting Concern Örnekleri

- **Logging** — Uygulamada gerçekleşen olayların kaydedilmesi.
- **Error Handling** — Hata yakalama, hata mesajlarının üretilmesi.
- **Authentication & Authorization** — Kullanıcı doğrulama ve yetkilendirme.
- **Caching** — Performans artırmak için veri saklama mekanizmaları.
- **Metrics & Monitoring** — Performans ve kullanım ölçümleri.
- **Security** — Veri doğrulama, girdi temizleme, saldırı önleme (XSS, CSRF vb.)
- **Data Validation** — Girilen verilerin doğrulanması.
- **Transaction Management** — Dağıtık sistemlerde tutarlılığı sağlama.
- **Internationalization (i18n)** — Çoklu dil desteği ve çeviri yönetimi.

Neden Cross-Cutting Concern Olarak Adlandırılır?

Bu sorumluluklar, tek bir modülde sınırlı kalmaz; genellikle aynı anda birçok bileşen, servis veya iş akışı tarafından paylaşılır. Bu nedenle sistem içinde “dikey” değil, “yatay” olarak yayılırlar:

İş mantığı katmanı, veri erişim katmanı ve sunum katmanı dahil olmak üzere pek çok yapıyı yatay biçimde etkilerler.

Problemler ve Motivasyon

Cross-cutting concern’ler doğrudan modüllerin içine yerleştirildiğinde:

- Kod tekrarı artar,
- Sınıfların sorumlulukları karışır (SRP ihlali),
- Test edilebilirlik azalır,

- Sistemin bakımı zorlaşır.

Bu nedenle modern mimarilerde cross-cutting concern'ler genellikle özel katmanlara taşınır.

Çözüm Yöntemleri

Cross-cutting concern'lerin doğrudan iş mantığı kodu içine eklenmesi, tekrar eden kod blokları, sorumluluk karmaşası (SRP ihlali) ve düşük bakım kolaylığı gibi sorunlara yol açar. Bu nedenle modern yazılım mimarisinde bu sorumlulukları soyutlamak için çeşitli teknikler kullanılır. Aşağıda dört temel yaklaşım detaylandırılmıştır.

1. Aspect-Oriented Programming (AOP) Aspect-Oriented Programming, cross-cutting concern'leri iş mantığından tamamen ayırmak için geliştirilmiş bir paradigmadır. Temel fikir, iş mantığı kodunun içine logging, güvenlik, hata yakalama veya transaction yönetimi gibi görevleri manuel olarak eklemek yerine, bu davranışları “aspect” adı verilen dış yapılara taşımaktır.

AOP şu temel bileşenlerden oluşur:

- **Join Point** — Kodun kesilebileceği noktalar (metot çağrılar, property erişimleri, exception fırlatma vb.)
- **Pointcut** — Hangi join point'lere müdahale edileceğini belirleyen kurallar.
- **Advice** — Müdahale sırasında çalışacak davranış (before, after, around).
- **Aspect** — Pointcut ve Advice kombinasyonu; cross-cutting concern'in modüler hâli.

Bu yaklaşım sayesinde iş mantığı “temiz” kalır; cross-cutting concern kodda görünmez ve otomatik olarak doğru noktalara uygulanır.

Aspect-Oriented Programming (AOP) Örnekleri

Aspect-Oriented Programming (AOP), iş mantığına dokunmadan, cross-cutting concern'leri kodun belirli noktalarına otomatik olarak enjekte etmeyi amaçlayan bir mimari yaklaşımdır. Aşağıda logging, authentication, transaction yönetimi ve performans ölçümü gibi tipik AOP kullanım senaryoları örnekleri verilmiştir.

1. Logging (Kayıt Tutma) Aspect'i

İş mantığına log ifadeleri yazmak, tekrarlı ve bakımı zor bir yaklaşım olduğundan AOP ile otomatik hâle getirilebilir.

Aspect Kodu

```
@Aspect()
class LoggingAspect {

    @Before("execution(* UserService.*(..))")
    logBeforeMethod() {
        console.log("Metot çalışmak üzere...");
    }

    @After("execution(* UserService.*(..))")
    logAfterMethod() {
        console.log("Metot tamamlandı.");
    }
}
```

Bu aspect, `UserService` içindeki tüm metodlara hiçbir iş mantığı kodu değişmeden logging davranışı ekler.

2. Authentication (Kimlik Doğrulama) Aspect'i

Kimlik doğrulama kontrolünün her metoda manuel eklenmesi yerine, AOP ile otomatik yapılması daha temiz ve sürdürülebilir bir çözümdür.

Aspect Kodu

```
@Aspect()
class AuthAspect {

    @Before("execution(* OrderService.*(..))")
    checkAuth() {
        if (!AuthService.isLoggedIn()) {
            throw new Error("User not authenticated");
        }
    }
}
```

Bu aspect, `OrderService` içindeki tüm metodların çalışmadan önce kimlik doğrulaması zorunluluğunu merkezileştirir.

3. Transaction Yönetimi Aspect'i

Transaction başlatma, commit etme ve hata durumunda rollback alma gibi işlemler iş mantığına karıştığında kod tekrarı ve karmaşıklık oluşur. AOP ile transaction yönetimi soyutlanabilir.

Aspect Kodu

```
@Aspect()
class TransactionAspect {

    @Before("execution(* OrderService.*(..))")
    beginTransaction() {
        db.begin();
    }

    @AfterReturning("execution(* OrderService.*(..))")
    commit() {
        db.commit();
    }

    @AfterThrowing("execution(* OrderService.*(..))")
    rollback() {
        db.rollback();
    }
}
```

Bu sayede `OrderService` metotları sadece iş mantığını içerir; transaction yönetimi tamamen dışarı taşınmıştır.

4. Performans Ölçümü (Timing) Aspect'i

Bir metodun kaç milisaniyede çalıştığını ölçmek için manuel zamanlayıcı koymak yerine AOP ile bu davranış tüm metotlara otomatik eklenebilir.

Aspect Kodu

```
@Aspect()
class PerformanceAspect {

    @Around("execution(* PaymentService.*(..))")
    measurePerformance(joinPoint) {
        const start = performance.now();
        const result = joinPoint.proceed();
        const end = performance.now();
        console.log(joinPoint.method + " " + (end-start) + "ms sürdü");
        return result;
    }
}
```

Bu approach ile `PaymentService` metodlarının performans analizi tamamen görünmez bir şekilde gerçekleştirilir.

2. Middleware Katmanları Özellikle web, API gateway veya sunuculu mimarilerde kullanılan middleware yaklaşımı, istek-yanıt akışı içinde araya katmanlar ekleyerek cross-cutting concern'leri merkezi bir noktada çözmeyi sağlar.

Bir request aşağıdaki gibi bir zincir üzerinden ilerler:

Client → Middleware(1) → Middleware(2) → ... → Handler (iş mantığı)

Her middleware şunları yapabilir:

- logging eklemek,
- authentication veya authorization kontrolü,
- rate limiting veya throttling,
- hata yakalama ve dönüştürme,
- token yenileme,
- yanıt veya istek verisini değiştirme.

Bu yöntem, sistemdeki tüm isteklerin tek bir akışta işlenmesini sağladığı için cross-cutting concern'lerin tekrarsız ve merkezi bir noktadan uygulanmasını mümkün kılar.

3. Global Servis veya Modüller Cross-cutting concern'lerin çözümü için sık kullanılan pratik bir yaklaşım, bu davranışları global servis ya da modüllerde toplamaktır. Bu servisler, uygulamanın herhangi bir yerinden erişilebilen ortak yapılardır.

Örnek global servisler:

- **LoggingService** — uygulama genelinde tutarlı log üretimi,
- **ErrorService** — hata mesajlarının standardizasyonu,
- **NotificationService** — kullanıcı bildirimlerinin yönetimi,
- **CacheService** — merkezi caching politikaları,
- **MetricsService** — performans ve kullanım ölçümleri.

Bu yöntem, AOP kadar soyut olmasa da, tekrar eden kod bloklarını azaltır ve iş mantığının cross-cutting logic içermesini engeller.

4. Decorator / Proxy Tasarım Kalıpları Bu yaklaşım, nesnelerin davranışlarını çalışırken genişletmek veya özelleştirmek için kullanılan nesne yönelimli tasarım kalıplarına dayanır. Bir sınıfın doğrudan değiştirilmesi yerine, o sınıfı “saran” bir dekoratör ile ek davranış sağlanır.

- **Decorator** — aynı arayüzü uygulayan ve orijinal nesneyi saran yapı. Ek davranış ekleyebilir.
- **Proxy** — asıl nesneye erişim kontrolü sağlayan, araya girerek davranışı yöneten yapı.

Bu yöntem şu senaryolar için idealdir:

- caching ekleme,
- retry politikası ekleme,
- logging davranışı ekleme,
- metrics toplama,
- authorization kontrolü yapma.

Bu yaklaşım **Open/Closed Principle** ile uyumludur: Sınıflar değişime kapalı, genişletilmeye açıktır.

Domain-Driven Design (DDD) ve Dil Bağımsız Domain Katmanı

Domain-Driven Design (DDD), karmaşık iş kurallarını taşıyan sistemlerde iş mantığının uygulama katmanlarından ayrıştırılarak **tek bir mantıksal merkezde** toplanmasını hedefleyen bir yazılım tasarım yaklaşımıdır. DDD'nin temel amacı, teknik detaylardan arındırılmış, saf bir “iş modeli” ortaya çıkarmak ve sistemin değişime uyum yeteneğini artırmaktır.

1. DDD'nin Amacı

DDD, şu problemlere çözüm üretir:

- İş kurallarının UI, API, Service ve Data katmanlarına dağılması,
- Kuralların tekrar etmesi (duplication) ve çelişkiler oluşturması,
- Kodun anlaşılabilir hâle gelmesi,
- İş mantığının genişledikçe karmaşılaşp kontrol edilememesi.

Bu sorunları çözmek için DDD şu prensibi getirir:

“Her domain nesnesi kendi kurallarını kendisi bilmeli ve uygulatmalıdır.”

Yani iş kuralları:

- UI'da,
- API Controller'da,
- Service katmanında,
- Veritabanı doğrulamalarında

bulunmamalıdır. Tüm kurallar **domain modelinin içinde** yaşamalıdır.

2. Domain Modeli ve Davranış Odaklı Tasarım

DDD'ye göre domain modeli, yalnızca veri tutan “anemik” yapılardan ibaret olmamalıdır. Gerçek bir domain modeli **davranış** içerir.

Örnek: Sipariş Domain Modeli

```
class Order {
  constructor(items) {
    this.items = items;
    this.status = 'PENDING';
  }

  pay() {
    if (this.status !== 'PENDING')
      throw new Error("Order already processed");
    this.status = 'PAID';
  }

  ship() {
    if (this.status !== 'PAID')
      throw new Error("Unpaid orders cannot be shipped");
    this.status = 'SHIPPED';
  }
}
```

Bu modelde iş kuralları:

- “Ödenmemiş sipariş kargoya verilemez.”
- “PENDING dışındaki sipariş ödenemez.”

gibi domain’e ait kurallar **modelin kendi içinde** tanımlanmıştır.

3. Uygulama Katmanlarının Rolü

DDD ele alındığında mimari roller şu şekilde ayrılır:

- **UI (Component):** Yalnızca görünüm ve kullanıcı etkileşimi.
- **Application/Service:** Orkestrasyon yapar fakat iş kuralı içermez.
- **Domain:** Saf iş kuralları ve davranışların merkezi.
- **Infrastructure:** Veritabanı ve dış bağımlılıklar.

Özellikle şu ayrım kritiktir:

UI “yap” der, domain “yapılır/yapılamaz” der.

4. Domain Katmanı Neden Dil Bağımsızdır?

Domain katmanı, iş kurallarının saf hâlini temsil ettiğinden, herhangi bir teknik detayla ilişkili değildir. İş kuralları, kullanılan programlama diliyle değil, *gerçek dünya davranışlarıyla* tanımlanır.

Örnek bir domain kuralı:

“Geçmiş tarihe randevu alnamaz.”

Bu kural:

- Java,
- TypeScript,
- Python,
- C#,
- Kotlin

gibi farklı dillerde aynı şekilde ifade edilir.

Ayrıca domain modeli şu teknolojileri bilmez:

- Angular, React, Vue (UI)
- NestJS, Spring Boot (backend)
- TypeORM, Prisma, Mongoose (ORM)
- HTTP protokolü, JSON, DTO'lar
- Observable, Promise, async işlemler

Bu nedenle domain katmanı:

- Dil bağımsızdır,
- Framework bağımsızdır,
- Platform bağımsızdır.

5. Angular Üzerinden DDD Örneği

Component (UI)

```
createAppointment(formValue) {  
  try {  
    this.appointmentService.createAppointment(formValue);  
  } catch (e) {  
    alert(e.message);  
  }  
}
```

Service (Orkestrasyon)

```
createAppointment(data) {  
  const appt = new Appointment(data.date, data.time);  
  appt.validate();          // Domain kuralı  
  return this.api.create(appt);  
}
```

Domain Modeli

```
class Appointment {  
  constructor(date, time) {  
    this.date = date;  
    this.time = time;  
  }  
  
  validate() {  
    if (this.date < new Date())  
      throw new Error("Geçmiş tarihe randevu alınamaz.");  
    if (this.isWeekend())  
      throw new Error("Hafta sonu randevu alınamaz.");  
  }  
  
  isWeekend() {  
    const d = this.date.getDay();  
    return d === 0 || d === 6;  
  }  
}
```

Domain modeli hiçbir şekilde Angular veya HTTP ile ilgili bilgi içermez. Sadece iş kurallarını taşır.

Sonuç

DDD, iş kurallarını “tek sorumlu” bir merkeze (domain katmanına) yerleştirerek uygulamanın her bölümünde tutarlılık ve sürdürülebilirlik sağlar. Bu katmanın programlama dili veya framework bağımlılığı olmadığı için, sistem tasarımı daha esnek ve genişletilebilir hale gelir.

DDD ve Domain-Driven Teams Arasındaki Fark

Domain-Driven Design (DDD) ile Spotify'nın "Domain-Driven Team" yaklaşımı, benzer terminoloji nedeniyle sıkça karıştırılmaktadır. Ancak bu iki kavram, farklı düzlemlerde işleyen, farklı problemlere çözüm sunan yapılardır. Bu bölümde farkları, ilişkileri ve etkileşim noktaları sistematik bir biçimde açıklanmaktadır.

Temel Fark: Mimari Tasarım vs. Organizasyon Tasarımı

- **DDD (Domain-Driven Design)** Yazılım mimarisi ve iş mantığının modellenmesi üzerine odaklanır. Amaç, karmaşık iş kurallarını *domain modelinde merkezileştirmek* ve teknik detaylardan arındırılmış bir iş mantığı oluşturabilmektir.
- **Domain-Driven Teams (Spotify Modeli)** Organizasyonel bir yapıdır. Ekiplerin, fonksiyonel veya teknik olarak değil, *iş domainlerine göre* yapılandırılmasını öngörür.

Bu nedenle:

DDD = Yazılımı nasıl tasarlırsın

Domain-Driven Teams = Ekibi nasıl organize edersin

Domain-Driven Team Nedir?

Spotify yaklaşımında ekipler (squads), belirli bir domain'in hem iş hem de teknik sorumluluğuna sahiptir. Örnek ekipler:

- Player Experience Team
- Search & Recommendation Team
- Payments Team
- Artist Management Team
- Playlist Team

Bu ekipler:

- frontend,
- backend,
- mobil,
- veri tabanı,
- operasyon,
- metrik takibi, roadmap

dahil olmak üzere **uçtan uca sorumluluk** taşırlar. Yani bir domain yalnızca kodda değil, organizasyonda da izole edilmiştir.

Bu yaklaşım, DDD'nin “bounded context” ilkesinin organizasyon yapısına uygulanmış hâli olarak düşünülebilir.

DDD ile Topolojinin Kesiştiği Noktalar

Her iki yaklaşım da şu prensiplerde buluşur:

- **Sınırların net tanımlanması** (DDD: bounded context Spotify: squad/tribe ayrımı)
- **Sahiplik (Ownership)** DDD'de kurallar domain içindedir. Spotify'da domain'e bir ekip sahiptir.
- **Bağımsız gelişebilme** Her domain kendi hızında ilerleyebilir.
- **Bağımlılıkların azaltılması**

Bu açıdan Spotify modeli, DDD'nin *organizasyonel bir yansımasıdır*.

Ayrışma Noktaları

DDD yazılım katmanlarını tanımlar:

- Domain Model
- Application Layer
- Infrastructure Layer
- UI Layer

Domain-Driven Teams ise organizasyon katmanlarını tanımlar:

- Squad
- Tribe
- Chapter
- Guild

Dolayısıyla teknik alan ile organizasyon alanı farklıdır.

Model-Driven Development (MDD) ve Prisma Örneği

1. MDD'nin Temel Fikri

Model-Driven Development (MDD), yazılım geliştirmede **modeli tek gerçek kaynak (single source of truth)** olarak kabul eden yaklaşımdır.

Koddan önce model tanımlanır; kod, bu modelden otomatik olarak üretilir.

Bu sayede aynı bilgiyi tekrarlayan şekilde DB şeması, TypeScript tipleri, API şemaları veya CRUD servislerinde yeniden yazmak gerekmez.

2. Temel Problem

Geleneksel geliştirme yaklaşımında bir model değiştiğinde aşağıdaki katmanlarda tekrar tekrar güncelleme yapmak gerekir:

- Veritabanı şeması (SQL)
- Backend model sınıfları
- API kontratları (OpenAPI/GraphQL)
- Frontend TypeScript tipleri
- CRUD servisleri ve validasyonlar

Bu durum hatalara, tutarsızlıklara ve bakım maliyetinin artmasına neden olur. MDD bu problemi çözmek için ortaya çıkmıştır.

3. Prisma ile MDD

Prisma, modeli tek merkezden tanımlayıp geri kalan her şeyi otomatik üreten modern bir MDD aracıdır.

3.1 Model Tanımı

Aşağıdaki Prisma modeli yazılır:

```
model User {
  id      Int      @id @default(autoincrement())
  name    String
  email   String   @unique
  age     Int?
}
```

Bu aşamada yalnızca **model** tanımlanmıştır; herhangi bir CRUD kodu, tip tanımı veya SQL yoktur.

3.2 Kod Üretimi

Aşağıdaki komut çalıştırılır:

```
npx prisma generate
```

Bu komut sonucunda Prisma aşağıdaki çıktıları otomatik olarak üretir:

4. Otomatik Üretilen Çıktılar

4.1 TypeScript Tipleri

```
export type User = {  
  id: number  
  name: string  
  email: string  
  age: number | null  
}
```

4.2 CRUD Metotları

```
prisma.user.findMany()  
prisma.user.findUnique({ where: { id: 1 } })  
prisma.user.create({ data: { name: "Ömer", email: "o@x.com", age: 30 } })  
prisma.user.update({ where: { id: 1 }, data: { age: 31 } })  
prisma.user.delete({ where: { id: 1 } })
```

4.3 SQL Şeması

```
CREATE TABLE "User" (  
  "id" INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,  
  "name" TEXT NOT NULL,  
  "email" TEXT NOT NULL UNIQUE,  
  "age" INTEGER  
);
```

4.4 Input Tipleri

```
export type UserCreateInput = {  
  name: string  
  email: string  
  age?: number | null  
}
```

```
export type UserUpdateInput = {  
  name?: string  
  email?: string  
  age?: number | null  
}
```


7. Event Delivery Guarantees (Teslimat Garantileri)

Amaç

Dağıtık sistemlerde event'lerin kuyruk veya mesajlaşma altyapısı üzerinden **nasıl teslim edileceği** kesin değildir. Bu nedenle Event-Driven mimariler üç farklı teslimat modelini destekler:

- **At-Most-Once**
- **At-Least-Once**
- **Exactly-Once**

Her modelin avantajları, riskleri ve kullanım alanları farklıdır.

1) At-Most-Once Delivery

Event en fazla bir kere teslim edilir. Teslimat kaybolabilir; fakat asla iki kere gönderilmez.

“Ya hiç gelmez, ya bir kez gelir.”

Avantajları

- Basit altyapı
- Duplicate işleme riski yok

Dezavantajları

- Kritik event'lerin kaybolma ihtimali vardır

Ne zaman kullanılır?

- Loglama
- Analitik event'leri
- Telemetry

2) At-Least-Once Delivery

Event en az bir kere teslim edilir. Duplicate gelebilir; fakat asla kaybolmaz.

“Kesin gelir, ama iki kere gelebilir.”

Avantajları

- Güvenilir teslimat
- Finansal ve operasyonel süreçler için güvenlidir

Dezavantajları

- Duplicate event işleme zorunluluğu
- Idempotency tasarımı gerekir

Ne zaman kullanılır?

- Ödeme akışları
- Stok yönetimi
- Mikroservis senkronizasyonu

3) Exactly-Once Delivery

Event ne kaybolur ne de birden fazla gelir. Teoride ideal modeldir; fakat pratikte doğrudan uygulanamaz.

“Ne eksik, ne fazla: tam bir kez işlenir.”

Gerçek Durum

Exactly-once, mesaj kuyrukları tarafından doğrudan sağlanmaz. Gerçekte aşağıdaki mekanizmaların birleşimiyle elde edilir:

- Idempotent iş mantığı
- Duplicate detection (eventId kontrolü)
- Transactional Outbox Pattern
- DB + Event Log atomik commit

Ne zaman kullanılır?

- Bankacılık işlemleri
- Para transferi
- Kritik state değişimleri

Bu Adımın Çıktısı

- Her event için hangi teslimat modelinin kullanılacağı belirlenir.
- Idempotency düzeyi netleştirilir.
- Event altyapısının gereksinimleri (retry, deduplication, outbox) planlanır.
- Bir sonraki aşama olan **Outbox Pattern ve Transactional Messaging** için temel oluşturur.

Driven Development (DD) Yaklaşımlarının Tam Listesi

Aşağıdaki bölüm, modern yazılım mühendisliğinde kullanılan tüm “Driven Development” yaklaşımlarını sistematik biçimde özetler. Her bir yaklaşım, yazılım geliştirme sürecini yönlendiren farklı bir odak noktasını temsil eder.

1. Test-Driven Development (TDD)

Önce testlerin yazıldığı, ardından bu testleri geçecek minimum kodun geliştirildiği, son olarak iyileştirme (refactor) yapılan yaklaşım.

- Döngü: **Test** → **Kod** → **Refactor**
- Amaç: Hata oranını azaltmak ve tasarımı sadeleştirmek.

2. Behavior-Driven Development (BDD)

Davranış odaklı geliştirme. Sistem davranışları Gherkin gibi doğal dillerle tanımlanır (*Given-When-Then*).

- İnsan-okur biçimli testler.
- PO-QA-Dev ortak anlayışı.

3. Acceptance Test-Driven Development (ATDD)

Geliştirmeye başlamadan önce kabul kriterlerinin tanımlanıp teste döküldüğü yaklaşım.

- “Kabul kriteri önce gelir.”
- Ürün ekipleriyle ortak çalışma.

4. Feature-Driven Development (FDD)

Geliştirme süreçlerinin “feature” odaklı yapılması.

- Feature tanımlama → Tasarım → Geliştirme.
- Feature listesi roadmap’in temelidir.

5. Domain-Driven Design (DDD)

Karmaşık iş alanlarında domain kurallarının tek bir merkezde toplandığı ve yazılımın iş mantığına göre şekillendirildiği yaklaşım.

- Bounded Context, Entity, Aggregate, Value Object.
- Kurallar domain modelinde yaşar.

6. Example-Driven Development (EDD)

Sistem davranışlarının örnekler üzerinden tanımlandığı yaklaşım.

- “Örnek $\rightarrow$ Kural $\rightarrow$ Kod” akışı.
- Kullanıcı örnekleri testlere dönüşür.

7. Story-Driven Development (SDD)

User story’lerin geliştirmeyi yönlendirdiği yaklaşım.

- “As a user...” formatı odaktır.

8. Responsibility-Driven Design (RDD)

Nesnelerin “sorumluluk” üzerinden modellenmesini savunan yaklaşım.

- SRP’nin kökeni.
- Nesnelerin görev alanları belirlenir.

9. Outcome-Driven Development (ODD)

Kod çıktısı değil, iş etkisinin önceliklendirildiği yaklaşım.

- “Sonuç odaklı” geliştirme.
- KPI / impact temelli tasarım.

10. Model-Driven Development (MDD)

Kodun modellerden üretildiği yaklaşım.

- UML $\rightarrow$ Kod üretimi.
- Model merkezli mimari.

11. Problem-Driven Development (PDD)

Çözümünden önce problemin netleştirildiği yaklaşım.

- Problem keşfi $\rightarrow$ çözüm tasarımı $\rightarrow$ geliştirme.

12. Intent-Driven Development (IDD)

Kodun “niyetinin” açıkça ifade edildiği yaklaşım.

- Okunabilirlik ve davranış şeffaflığı.

13. Contract-Driven Development (CDD)

Özellikle mikroservislerde tüketici odaklı API sözleşmeleri ile geliştirme yapılması (Pact).

- “Consumer-driven contracts”
- API sözleşmesi geliştirmeyi yönlendirir.

14. Data-Driven Development

Kararların veri analizine ve deney sonuçlarına göre verildiği yaklaşım.

- A/B testi, metrik odaklı karar alma.

15. Event-Driven Development

Sistemin davranışlarının event’ler tarafından yönlendirildiği yaklaşım.

- Event sourcing, CQRS, messaging.

16. Risk-Driven Development

Önceliklendirmeyi teknik veya iş risklerine göre yapan yaklaşım.

- En riskli bölümler önce geliştirilir.

17. Learning-Driven Development

Koddan önce “öğrenmeyi” optimize etmeyi amaçlayan yaklaşım.

- MVP → kullanıcı geri bildirimi → öğrenme döngüsü.

Sonuç

Driven Development yaklaşımları, yazılım geliştirmeyi farklı odak noktalarına göre yönlendirir. Her yaklaşımın amacı, süreçleri daha öngörülebilir, güvenilir ve etkili hale getirmektir.

Frontend’te Consistency ve State Management

Bu bölümde, modern frontend uygulamalarının dağıtık sistemlerle etkileşiminde sıkça karşılaşılan dört temel kavram ele alınmaktadır. Aggregate Design, Saga Pattern, CAP Teoremi ve Causal Consistency; özellikle gerçek zamanlı veri akışları, çok adımlı işlemler, optimistic UI güncellemeleri ve state tutarlılığı gibi konularda frontend geliştiricilere doğrudan yol gösteren mimari prensiplerdir.

Aggregate Design

DDD’de **Aggregate**, tek bir *tutarlılık sınırı* (consistency boundary) olarak tanımlanır. Frontend açısından Aggregate; bir ekranın veya domain nesnesinin **durum geçişlerini**, **iş kurallarını** ve **validasyonlarını** tek bir bütün olarak modellemek demektir. Örneğin bir randevu nesnesi şu davranışlara sahiptir:

```
confirm(), cancel(), reschedule(), markPaid()
```

UI tarafında geçişler:

- PAID değilse `confirm()` yapılamaz.
- CANCELLED ise `reschedule()` yapılamaz.
- CONFIRMED ise `cancel()` mümkündür.

Bu kuralların kod boyunca dağılmak yerine tek bir agregada toplanması, frontend tarafında **tutarlı bir ekran davranışı** sağlar.

```
class Appointment {
  constructor(status) {
    this.status = status; // 'PENDING' | 'PAID' | 'CONFIRMED' | 'CANCELLED'
  }

  canCancel() {
    return this.status === 'PENDING' || this.status === 'CONFIRMED';
  }

  canConfirm() {
    return this.status === 'PAID';
  }

  canReschedule() {
    return this.status !== 'CANCELLED';
  }
}

// UI kullanım:
<button disabled={!appointment.canCancel()}>Cancel</button>
```

Saga Pattern

Saga, backend'de dağıtık işlemleri adım adım yöneten bir orkestrasyon modelidir. Frontend'de Saga; **çok adımlı asenkron UI akışlarının yönetimi** anlamına gelir. Her adım bir "yerel işlem"dir; hata olursa UI geri sarar:

```
upload → convert → createMetadata → finalize
                      rollback on error
```

Saga, frontend'de **orchestration, retry, rollback** ve **state machine** mantığının temelidir.

```
// NgRx Effects gibi zincirli bir akışı temsil eder:
```

```
uploadVideo$ = createEffect(() =>
  this.actions$.pipe(
    ofType(startUpload),
    switchMap(action => this.api.upload(action.file)),
    switchMap(fileId => this.api.convert(fileId)),
    switchMap(result => this.api.createMetadata(result)),
    map(metadata => finalizeSuccess({ metadata })),
    catchError(() => of(rollback()))
  )
);

// UI sadece tek bir aksiyon tetikler:
this.store.dispatch(startUpload({ file }));
```

Saga Pattern: Çok Adımlı UI Akışı ve Rollback

Aşağıdaki örnek, bir dosya yükleme sürecinin (upload → convert → metadata → finalize) NgRx Effects ile nasıl Saga benzeri bir yapıda yönetildiğini göstermektedir. Herhangi bir adım hata verdiğinde süreç tamamen iptal edilmekte ve `rollback()` aksiyonu tetiklenmektedir.

```
// Saga benzeri çok adımlı async akış (NgRx Effects)
```

```
uploadFlow$ = createEffect(() =>
  this.actions$.pipe(
    ofType(startUpload),
    // 1. Adım: Upload
    switchMap(action => this.api.upload(action.file)),

    // 2. Adım: Convert
    switchMap(fileId => this.api.convert(fileId)),

    // 3. Adım: Metadata
    switchMap(result => this.api.createMetadata(result)),
```



```

    // 4. Adım: Finalize
    map(metadata => finalizeSuccess({ metadata })),

    // Herhangi bir adımda hata olursa rollback tetiklenir
    catchError(() => of(rollback()))
  )
);

// UI yalnızca akışı başlatır:
this.store.dispatch(startUpload({ file }));

```

Rollback Reducer Örneği

```

on(rollback, (state) => ({
  ...state,
  isUploading: false,
  video: null,
  converted: null,
  metadata: null,
  error: 'Upload process failed. All steps reverted.'
}));

```

Özet

Bu yaklaşım, çok adımlı bir UI sürecinde: - başarısızlık durumunda yarım kalmış state'i temizler, - optimistic güncellemeleri geri alır, - UI'yı tutarlı bir duruma geri döndürür. Saga Pattern'ın frontend karşılığı olan bu yapı, kompleks async akışları tek noktadan yönetilebilir hale getirir.

CAP Theorem

CAP Teoremi (Consistency, Availability, Partition Tolerance), dağıtık sistemlerde bu üç özelliğin aynı anda tam olarak sağlanamayacağını ifade eder. Frontend açısından bu, UI'nın her zaman backend ile birebir senkron kalamayacağı anlamına gelir. Bu durum özellikle cache kullanımında, gerçek zamanlı veri akışlarında ve optimistic update senaryolarında doğrudan görünür hale gelir.

Frontend Üzerindeki Etkiler

- **Stale UI:** UI cache'i backend güncellemelerinin gerisinde kalabilir.
- **Optimistic Update Rollback:** UI anında günceller; backend reddederse değişiklik geri alınır.
- **Concurrent Updates:** Bir kaydı aynı anda düzenleyen kullanıcılar tutarsız sonuçlarla karşılaşabilir.
- **Offline Mode:** Kullanıcı çevrimdışıyken yalnızca eski veri gösterilebilir.
- **Cache Tutarsızlığı:** SWR, TanStack Query ve NgRx Query gibi araçlar, CAP'in yarattığı tutarsızlık sorunlarını azaltmak için tasarlanmıştır.

UI her zaman *tam tutarlılık* sunamaz; bu doğal bir sonuçtur. Bu nedenle frontend mimarileri aşağıdaki yaklaşımları kullanarak tutarsızlığı kontrol altında tutar:

- **Background Revalidation:** UI ilk olarak mevcut (muhtemelen eski) veriyi gösterir, ardından arka planda güncel veriyi getirerek sessizce ekranı yeniler. Böylece hem hızlı yükleme hem de güncel veri sağlanmış olur.
- **Stale-While-Revalidate:** Kullanıcıya hemen "eski" cache verisi gösterilir; aynı anda yeni veri talebi yapılır. Backend'den güncel veri geldiğinde UI otomatik olarak güncellenir. Bu yaklaşım, hızlı ilk render ve gecikmesiz kullanıcı deneyimi sağlar.
- **Optimistic Update ve Rollback:** UI, backend yanıtını beklemeden değişikliği anında uygular (optimistic update). Backend işlemi başarısız olursa UI eski hâline döner (rollback). Bu yöntem, kullanıcıya hızlı ve akıcı bir etkileşim hissi verirken hata durumlarında tutarlılığı korur.

Causal Consistency

Causal Consistency, olayların sistemde gerçekleştiği sıranın; UI'da da **aynı sırayla görünmesi** gerektiğini söyler. Özellikle gerçek zamanlı uygulamalarda kritik bir problemidir.

Frontend Senaryosu

Kullanıcı şu işlemleri yapar:

1. Mesaj gönderir (A)
2. Mesajı beğenir (B)
3. Mesajı siler (C)

Ancak WebSocket eventleri şu sırayla gelebilir:

$$C \rightarrow A \rightarrow B$$

Bu durumda:

- Silinen mesaj geri gelir
- Sonra tekrar silinir
- Ardından like bilgisi düşer

UI tamamen tutarsız bir hale gelir. Bu, **event ordering** problemidir.

Çözüm

- Event zaman damgası karşılaştırma (timestamp)
- Sıra numarası (sequence number)
- Event buffering
- State machine ile geçiş kuralları

Özet

Causal Consistency, gerçek zamanlı UI'larda (chat, feed, notification) **doğru sıralı state güncellemeleri** için zorunlu bilgidir.

```
// WebSocket'ten gelen event'leri sıraya göre işleme:
```

```
let lastTimestamp = 0;
```

```
function handleEvent(event) {
```

```
if (event.timestamp < lastTimestamp) {  
    // Geç gelen event -> buffer'a al  
    buffer.push(event);  
    return;  
}  
  
// Doğru sıradaki event  
lastTimestamp = event.timestamp;  
applyToUI(event);  
}
```

Global Error Handling in Angular

1. Giriş

Modern Angular uygulamalarında hem **frontend (runtime)** hem de **backend (HTTP)** kaynaklı hataların ortaya çıkması kaçınılmazdır. Bu nedenle hataları **tek bir merkezi noktada** toplamak ve kullanıcıya tutarlı geri bildirim sağlamak kritik bir mimaridir.

Bu doküman, Angular'da global hata yakalamayı aşağıdaki iki temel yapı üzerinden açıklar:

- **GlobalErrorHandler** — uygulama içerisindeki tüm runtime hatalarını yakalar.
- **HTTP Interceptor** — backend isteklerindeki hataları ve loading davranışını yönetir.

2. Hata Türleri

2.1 Local Frontend Errors

Angular component, service veya template içerisinde oluşan tüm runtime hatalardır:

- `throw new Error()`
- Component lifecycle hataları
- RxJS error akışları
- Template render hataları

Bu hatalar, Angular'ın **ErrorHandler** mekanizması tarafından globalde yakalanabilir.

2.2 Backend (HTTP) Errors

Backend'den dönen 4xx / 5xx statü kodları:

- 400, 401, 403, 404
- 500, 502, 503
- Timeout, Network Error

Bu tip hatalar **HTTP Interceptor** içerisinde yönetilir ve istenirse **ErrorHandler** içerisine yönlendirilebilir.

3. Mimarinin Genel Yapısı

- **core module:** Global error handler ve interceptor'lar burada tanımlanır.
- **shared module:** UI bileşenleri (loading spinner, error dialog).
- **app module:** Giriş modülü.

```
src
  core
    errors
      global-error-handler.ts
      http-loading.interceptor.ts
    core.module.ts
  shared
    errors
      error-dialog.service.ts
    loading
      loading-dialog.service.ts
    shared.module.ts
  app.module.ts
```

4. Global Error Handler

Amaç

Uygulamada herhangi bir yerde fırlatılan hatayı **tek bir noktadan** yakalamak ve kullanıcıya anlamlı bir mesaj sunmak.

Örnek Kod

```
@Injectable()
export class GlobalErrorHandler implements ErrorHandler {
  constructor(private dialog: ErrorDialogService, private zone: NgZone) {}

  handleError(error: any): void {
    console.error('GLOBAL ERROR:', error);

    this.zone.run(() => {
      this.dialog.openDialog(error.message || 'Unknown error');
    });
  }
}
```

Çalışma Prensibi

- Tüm runtime hatalarını otomatik yakalar.
- Dialog göstererek UI'ı tekrar kontrol altına alır.
- Loglama / Telemetry (Sentry vb.) için tek noktadır.
- Zone dışında oluşan hataları da yönetebilmek için NgZone kullanılır.

5. HTTP Loading Interceptor

Amaç

Her HTTP isteğinde:

1. Loading spinner açmak
2. Request'i çalıştırmak
3. Hata olsa bile spinner'ı kapatmak

Örnek Kod

```
@Injectable()
export class HttpLoadingInterceptor implements HttpInterceptor {
  constructor(private loading: LoadingDialogService) {}

  intercept(req: HttpRequest<any>, next: HttpHandler) {
    this.loading.openDialog();

    return next.handle(req).pipe(
      finalize(() => this.loading.closeDialog())
    );
  }
}
```

Açıklama

- `finalize()` operatörü hem success hem error durumunda tetiklenir.
- Error durumunda `GlobalErrorHandler` tetiklenir.
- Loading göstergesi uygulama çapında tamamen otomatikleşmiş olur.

6. Sonuç

Bu mimari:

- Tüm hataların tek noktada işlenmesini sağlar.
- Kullanıcıya tutarlı hata mesajları sunar.
- UI donmalarını ve belirsiz davranışları ortadan kaldırır.
- Backende yapılan tüm isteklerde otomatik loading davranışı oluşturur.
- Loglama ve monitoring (Sentry gibi) entegrasyonunu çok kolaylaştırır.

Sonuç olarak: Angular uygulamalarında global error handler + HTTP interceptor yapısı, tüm hata yönetimini sürdürülebilir ve merkezi bir hale getirir.

Global Error Tracking Libraries

Bu bölüm, web, mobil ve backend uygulamalarında oluşan tüm hataları **global olarak yakalamak, izlemek ve raporlamak** için kullanılan araçların kapsamlı bir listesini içerir.

1. Sector Standards (Most Popular)

Aşağıdaki araçlar, global crash ve error tracking alanında en yaygın kullanılan çözümlerdir:

- **Sentry** — Web, Mobile, Backend için en kapsamlı hata izleme sistemi.
- **Firebase Crashlytics** — Mobil uygulama crash takibi için endüstri standardı.
- **Bugsnag** — Web ve mobil uygulamalar için güçlü bir crash analizi platformu.

2. Web-Oriented Error Tracking Tools

Web uygulamalarında global hata yakalama ve raporlama için kullanılan araçlar:

- **Sentry (Web SDK)**
- **Bugsnag Web**
- **Rollbar**
- **Raygun**
- **LogRocket** (Session Replay + Error Tracking)
- **TrackJS**
- **Honeybadger**
- **Airbrake**

3. Backend Error Tracking Tools

Node.js, API ve microservice altyapılarında kullanılan global hata izleme çözümleri:

- **Sentry (Node SDK)**
- **Bugsnag for Node.js**
- **Airbrake**
- **Rollbar Node SDK**
- **Elastic APM**

- **Datadog APM**
- **New Relic**
- **OpenTelemetry (OTel)**

4. Mobile Crash Reporting Tools

React Native, Android ve iOS uygulamalarında kullanılan global crash yakalama araçları:

- **Firebase Crashlytics**
- **Sentry React Native**
- **Bugsnag Mobile**
- **Instabug**
- **AppCenter Diagnostics**
- **Raygun Mobile**
- **Embrace.io**

5. Logging + Error Tracking Hybrid Systems

Hem loglama hem de hata izleme fonksiyonlarını bir arada sağlayan kapsamlı çözümler:

- **Datadog**
- **New Relic**
- **Elastic APM**
- **Grafana Loki + Tempo**
- **Splunk**
- **LogDNA**

6. Özet

- **Sentry** — Web + Mobil + Backend için en iyi tümleşik çözüm.
- **Crashlytics** — Mobil uygulamalar için en iyi native crash raporlama.
- **LogRocket** — Kullanıcı oturumlarını video olarak kaydederek hata tespiti kolaylaştırır.
- **Datadog / New Relic** — Enterprise seviyesinde error + log + performance takibi.

Null Kontrollerinin Doğru Katmanda Yapılması

Modern bir frontend/backend mimarisinde, dış sistemlerden (API, DB, Cache) gelen veriler her zaman güvenilir kabul edilir. Bu nedenle null kontrollerinin **her katmanda değil**, yalnızca **doğru sınır katmanlarında** yapılması gerekir.

Bu doküman, null/undefined değerlerinin hangi katmanda nasıl ele alınması gerektiğini özetlemektedir.

1. Transport Layer (Interceptor / Network Boundary)

Bu katman sistemin dış dünya ile ilk temas noktasıdır. HTTP istekleri ve yanıtları burada ele geçirilir.

Bu katmandaki amaç:

- Response gövdesinin (body) boş gelmesi,
- Beklenmeyen format (örneğin string yerine object),
- Eksik veya çökmüş JSON,
- Network katmanındaki tutarsızlıklar

gibi sorunlarda **format düzeltme (transport normalization)** yapmaktır.

Örnek

```
body = body ?? {};  
body.data = body.data ?? null;  
body.message = body.message ?? "";
```

Bu katmanda **domain objelerine dokunulmaz**. Sadece “HTTP düzeyinde temiz bir zarf” oluşturulur.

2. Data Access Layer (Repository / API Service)

Bu katmanın amacı:

- DTO’yu domain modeline dönüştürmek,
- Field bazlı null kontrolü yapmak,
- Eksik alanlara default değer atamak,
- Web API’den gelen yapıyı dahili modele map etmek

Bu işlem **mapping ve normalization** işidir.

Örnek

```
function normalizeUser(dto: UserDto): User {  
  return {  
    id: dto?.id ?? "",  
    name: dto?.name ?? "Unknown",  
    age: dto?.age ?? 0,  
    settings: dto?.settings ?? { theme: "light" }  
  };  
}
```

Bu katmanın görevi, UI'ye hiçbir zaman null yansıtmayan **garantili bir domain modeli** üretmektir.

3. Application Layer (Use Cases / Services)

Bu katmanda null kontrolü yalnızca:

- kritik iş kurallarında,
- domain'in çalışması için mutlaka gerekli alanlarda,
- guard clause mantığında

yapılır.

Örnek

```
if (!user.id) throw new Error("Invalid user state");
```

Buradaki null kontrolleri yalnızca **iş kuralı ihlallerini engellemek** içindir.

4. UI Layer (Component / View)

UI katmanında null kontrolü **yapılmamalıdır**. Çünkü alt katmanlar gerekli temizliği çoktan yapmış olmalıdır.

UI sadece **saf, normalize edilmiş** veri ile çalışır.

Yanlış Örnek (Anti-Pattern)

```
<p>{{ user?.settings?.theme ?? "light" }}</p>
```

Doğru Örnek

```
<p>{{ user.theme }}</p>
```

Genel Özet

- **Interceptor (Transport Layer):** Response formatını normalize eder.
- **API Service / Repository:** Field-level null normalization ve mapping.
- **Application Layer:** İş kuralı odaklı guard clause.
- **UI Layer:** Null kontrolü gerekmez (temiz model kullanır).

Bu yaklaşım; Clean Architecture, DDD ve cebirsel veri tipleri prensiplerine uygun, sürdürülebilir ve yüksek güvenilirlik sağlayan bir veri akışı oluşturur.

Sonuç

First principle thinking, yazılım geliştirmede “hangi teknoloji daha popüler?” sorusundan çok daha önce gelen bir zihinsel disiplindir. Problemi yüzeyde görünen şekliyle değil, onu oluşturan temel unsurlar ve kısıtlar üzerinden düşünmeye zorlar. Bu sayede ezbere çözüm kalıpları yerine, bağlama uygun, gerekçelendirilmiş ve daha sade çözümler üretmek mümkün olur.

Pratikte bu yaklaşım; karmaşık bir bug incelerken “Gerçekte ne yanlış gidiyor?”, bir mimari karar verirken “Asgari hangi bileşenler olmadan bu sistem çalışmaz?” veya performans problemi çözerken “Neyi ölçüyorum ve neden?” gibi soruları alışkanlık hâline getirmekle başlar. Zamanla, kopyala-yapıştır çözümler yerine, kendi mantığını kuran ve aldığı her kararı açıklayabilen bir mühendis profili ortaya çıkar.

Sonuç olarak bu doküman, belirli bir teknoloji yığımına bağlı olmayan; fakat hangi dili veya çerçeveyi kullanırsan kullan, arka planda seni taşıyabilecek bir düşünme kasını güçlendirmeyi amaçlar. Eğer burada anlatılan prensipleri küçük adımlarla günlük işlerine taşıyabilirsen, sadece daha iyi kod yazmakla kalmaz; problemleri tanımlama, soyutlama ve iletişim kurma biçimin de doğal olarak evrilir. Bu da uzun vadede seni “daha çok araç bilen” bir geliştiriciden ziyade, “daha iyi düşünen” bir problem çözücü hâline getirir.